

Final Report

Generating and Exploring Feature Level-Sets in Arbitrary Multi-field Datasets

Michéal Nestor

Submitted in accordance with the requirements for the degree of
BSc Computer Science

2023/24

COMP3931 Individual Project

The candidate confirms that the following have been submitted.

Items	Format	Recipient(s) and Date
Final Report	PDF file	Uploaded to Minerva (08/05/24)
Project Showcase Video	URL	Sent to supervisor and assessor (08/05/24)
Link to online code repository	URL	Sent to supervisor and assessor (08/05/24)
Build and usage instructions	README file	Included in repository sent to supervisor and assessor (08/05/24)

The candidate confirms that the work submitted is their own and the appropriate credit has been given where reference has been made to the work of others.

I understand that failure to attribute material which is obtained from another source may be considered as plagiarism.

(Signature of Student) Michéal Nestor

Summary

This project aims to reproduce Jankowai and Hotz' [12] work in rendering data structures called feature level-sets in mutli-field datasets. This will be in the form of a tool which will allow a user to load arbitrary multi-fields datasets and generate feature level-set renders. This report will propose an architecture for such a tool, and will document our process for implementing this architecture. It will also discuss related research areas, alternative approaches to mutlivariate data visualisation and identify areas for future research and development.

Acknowledgements

I would like to firstly thank my incredible girlfriend Niamh Houston who has given me strength and been vital to my effective management of the stress and workload over the last few months. I would like to thank my parents for their unwavering support and constant and infectious positive attitude. I would like to thank Professor Hamish Carr for being an amazing supervisor, helping improve my self confidence, ensuring that the project remained on track through our weekly meetings and supporting my research by personally tutoring me on some of the complex theory behind this project.

I would also like to thank Bill Kuo, Wei Wang, Cindy Bruyere, Tim Scheitlin, and Don Middleton of the U.S. National Center for Atmospheric Research (NCAR), and the U.S. National Science Foundation (NSF) for providing the Weather Research and Forecasting (WRF) Model simulation data of Hurricane Isabel.

Contents

1	Introduction and Background Research	1
1.1	Introduction	1
1.2	Key Terminology and Fundamental Concepts	1
1.2.1	Multi-Field Datasets	1
1.2.2	Contours and Iso-Surfaces	1
1.2.3	Fibers and Fiber Surfaces	2
1.2.4	Traits, Features and Feature Level-Sets	2
1.2.5	Distance Field Generation	2
1.2.6	Trait Definition	3
1.2.7	Rendering Techniques for Iso-Surfaces	3
1.2.8	Marching Cubes Algorithm	3
1.3	Related Research and Alternative Approaches	4
2	Methods	6
2.1	Solution Architecture	6
2.1.1	3D Environment	7
2.1.2	Dataset Reader	8
2.1.3	User Interface	8
2.1.4	Distance Field Generation	9
2.1.5	Marching Cubes Algorithm	9
2.1.6	Program Data Controller	9
2.2	Pre-Development Decisions	10
2.2.1	Version control	10
2.2.2	Graphics API & Function Loader	10
2.2.3	Language	10
2.2.4	Testing	11
2.2.5	Build Systems	11
2.2.6	User Interface Libraries	12
2.2.7	Other Libraries	12
2.2.8	The Dataset	13
2.3	Development Methodology	13
2.3.1	Project Setup	14
3	Implementation	15
3.1	Implementation	15
3.1.1	Program Data Controller	15
3.1.2	3D Environment	15
3.1.3	Dataset Reader	17
3.1.4	Marching Cubes	18

3.1.5	Graphical User Interface	19
3.1.6	Distance Field Generation	19
3.1.7	Surface Rendering	20
4	Results and Discussion	21
4.1	Rendering Feature Level-Sets	21
4.2	Measuring Success	22
4.2.1	Efficacy	22
4.2.2	Performance	25
4.3	Future Development	26
4.3.1	Separating Distance Fields from Feature Level-Sets	26
4.3.2	3D Environment	26
4.3.3	Distance Field Generation	27
4.3.4	Marching Cubes	27
4.3.5	Dataset Reader	28
4.3.6	User Interface	28
4.4	Future Research	28
4.4.1	Measuring Euclidean Distance to a Trait	28
4.4.2	Euclidean Distance as a Metric	29
	References	30
	Appendices	32
A	Self-appraisal	32
A.1	Critical Self-Evaluation	32
A.2	Personal Reflection and Lessons Learned	33
A.3	Legal, Social, Ethical and Professional Issues	33
A.3.1	Legal Issues	33
A.3.2	Social Issues	34
A.3.3	Ethical Issues	34
A.3.4	Professional Issues	34
B	External Material	35
C	UML Diagrams	36
D	Code Snippets	41
D.1	Generating the Velocity Derived Attribute	41
D.2	Generating Attribute Histograms	42

Chapter 1

Introduction and Background Research

1.1 Introduction

Feature level-sets are data structures introduced by Jankowai and Hotz [12] that highlight regions in space with shared characteristics. These level-sets facilitate the extraction of features from, and thus the analysis of, multivariate spatial datasets. This project aims to replicate the work of Jankowai and Hotz by producing visualisation software that allows users to load generic multivariate spatial datasets, create and render feature level-sets in real-time, and add to their work by providing a first-person camera with which users can explore their renders. We will begin this chapter with definitions of key terminology, then move on to explore related research and compare alternative approaches to the visualisation of these datasets. Later chapters will define an architecture for a solution that realises the project's aim, describe how we implemented such an architecture, discuss our results and opportunities for future research and development.

1.2 Key Terminology and Fundamental Concepts

1.2.1 Multi-Field Datasets

With the lack of consistent terminology in the field for describing mutli-variate spatial datasets, a definition of the specific datasets we are focused on is required. Defined by Hotz [10], a multi-field is a set of fields $\{F_1, F_2, \dots, F_r\}, r \in \mathbb{N}$. Each field describes a function $F_i : D_i \rightarrow R_i$. For this project we follow the example of Jankowai and Hotz [12] and assume to encounter datasets with continuous fields and impose a restriction of D_i to $D_i = D \subset \mathbb{R}^3$, we expect just a three dimensional spatial domain without a temporal or parameter components included in Hotz's original definition [10]. We also impose a restriction of R_i , further than Jankowai and Hotz [12], to be a set of scalars such that $R_i \subset \mathbb{R}$. Like Jankowai and Hotz [12] we introduce the concept of attribute space $\mathcal{A}$ which refers to the fields of interest extended by derived quantities. Therefore, attribute space can be summarised in the mapping $f : D \rightarrow \mathcal{A} \in \mathbb{R}^n$ where n is the number fields of interest plus the number of derived quantities.

1.2.2 Contours and Iso-Surfaces

Given a function $f : \mathbb{R}^2 \rightarrow \mathbb{R}$ which map values in two dimensions to values in a single dimension, a contour can be defined as the inverse image $f^{-1}(h) = \{x \in \mathbb{R}^2 | f(x) = h\}$ for some iso-value $h \in \mathbb{R}$. Contours can be used to show points on a map which share a common height value, and allow a user to gain an understanding of the topology of the area mapped. Iso-surfaces are extensions of contours into a three dimensional domain, for a function $f : \mathbb{R}^3 \rightarrow \mathbb{R}$ they are defined similarly as the inverse image $f^{-1}(h) = \{x \in \mathbb{R}^3 | f(x) = h\}$ for some iso-value $h \in \mathbb{R}$. Iso-surfaces, like contours, allow users to gain a topological understanding of how some scalar varies in a given space. Iso-surfaces and contours can both be described as having one dimensional

attribute space $\mathcal{A}$.

1.2.3 Fibers and Fiber Surfaces

Fibers [6] result from extending the range of the scalar function that defines iso-surfaces into two dimensions, such a function is of the form $f : \mathbb{R}^3 \rightarrow \mathbb{R}^2$. A fiber then, is the inverse image $f^{-1}(c, d) = \{x \in \mathbb{R}^3 | f(x) = (c, d)\}$. It describes a region of overlap of attribute values c and d in the spatial domain. A practical example of a fiber is the areas in a room where both temperature is 18°C and air pressure is 10Pa. Fiber surfaces [6] are defined as the inverse image $f^{-1}(P) \in \mathbb{R}^3$ of any path $P \in \mathbb{R}^2$. Fibers can be defined as having two dimensional attribute space $\mathcal{A}$ and fiber surfaces can be described as paths in attribute space. Fiber surfaces allow for geometries to be defined in $\mathbb{R}^2$, this is especially powerful when paired with a continuous scatter-plot in $\mathbb{R}^2$ showing the user the distribution of values, thus allowing a user to identify features in $\mathbb{R}^2$, and with the corresponding fiber surface see their inverse image in $\mathbb{R}^3$.

1.2.4 Traits, Features and Feature Level-Sets

Generally a feature is the inverse image in the spatial domain D of some trait defined in attribute space $\mathcal{A}$. In terms of fiber surfaces, the fiber surface itself is a feature, and the path $P \in \mathcal{A}$ is the trait. As Jankwai and Hotz do [12] to deal with the problem of empty features, we consider the distance to traits in attribute space $\mathcal{A}$. This allows for the definition of the trait distance field $d_T : \mathcal{A} \rightarrow \mathbb{R}$ which assigns a scalar distance, or level, to each point $a \in \mathcal{A}$, $d_T(a) = \min\{\|a - t\| \mid t \in T\}$ where $\|\cdot\|$ represents the Euclidean norm in attribute space $\mathcal{A}$. From the trait distance field d_T the feature distance field can be defined $d_F = d_T \circ f$ as the pull-back of the trait distance field into the spatial domain D . This allows each point in $d \in D$ to have some value $d_F(d)$ which maps to the Euclidean distance of its image to the defined trait in attribute space $\mathcal{A}$. At last, feature level-sets f_c can be defined as in [12] as the set of points $d \in D$ that share level c , $d_F^{-1}(c) = \{d \in D \mid d_F(d) = c\}$. The zero feature level-set corresponds to the feature in spatial domain matching the trait exactly. The smallest non-empty feature level-set shows the points in the domain that are closest to the feature in attribute space.

1.2.5 Distance Field Generation

As previously stated, the feature distance field stores for each point in the dataset, the minimum distance to the trait of that points image in attribute space. For attribute spaces consisting of one, two or three dimensions, this calculation is straight forward using vector based geometric calculations, however, moving onto higher dimensions this calculation becomes increasingly more challenging. Jankowai and Hotz [12] describe distance field calculation for traits defined as a simple set of points but neglect to mention how the distance field would be calculated if those points were connected to describe a single trait geometry. An approach that could be used for geometric traits defined in arbitrary dimensions, is generating a convex hull (smallest convex set that contains the points) of the trait's vertices using the beneath-beyond method [9], then for each point calculate the distance to each hyper-plane which bounds the convex hull [2], taking the minimum as the distance to the trait. The computational requirements for computing distance fields for traits increase exponentially with the dimensionality of the trait.

1.2.6 Trait Definition

Trait definition requires a mechanism to select areas of interest in attribute space. For the case of fiber surfaces a continuous scatter plot of the two attributes where the user can manually draw the trait on the plot has been shown to be effective [6], this approach would also be effective for three dimensional attribute spaces. However, when the dimensionality of attribute space exceeds three, as is possible with multi-field datasets, a scatter plot becomes ineffective. With the goal of a definition system which works for n dimensional datasets Jankowai and Hotz [12] propose two approaches applicable to generic multi-field datasets: connecting selected points in attribute space, and using parallel coordinates. Connecting selected points requires the user to define one or more coordinates in attribute space and parallel coordinates require each dimension of attribute space is assigned to a parallel axis, then users can select either ranges or points on the parallel axis, both methods result in a geometric trait defined in attribute space.

1.2.7 Rendering Techniques for Iso-Surfaces

Feature level-sets can be thought of as iso-surfaces with each point's distance in attribute space from the trait being its iso-value, and the distance c for the feature level-set being the iso-value which generates the corresponding surface. Therefore, Jankowai and Hotz [12] propose geometric rendering as one of two options for rendering feature level-sets, they also suggest the option of direct volume rendering using raycasting and a transfer function editor. Direct volume rendering, originally introduced by Drebin et al. [8], creates images directly from volume data without first converting that data to a surface from which a mesh can be created and rendered. Due to the complexity of this project we could only implement one of the rendering techniques. Geometric rendering was chosen because of the improved performance for the user when exploring the feature level-sets with the first person camera. After an iso-surface has been extracted, rendering the resulting mesh is computationally cheap; however, for direct volume rendering, each time the camera's perspective is adjusted the expensive raycasting calculations required to create the volumes have to be recalculated.

1.2.8 Marching Cubes Algorithm

Thanks to both its simplicity and robustness the marching cubes algorithm [14] is the primary tool used for creating geometric renders of iso-surfaces [23]. Marching cubes is most easily explained in a two dimensional context, Figure 1.1 shows a curve in red and the curve resulting from marching cubes in blue. The algorithm works by splitting the domain into cubes (in two dimensions squares), and then comparing each cube independently with a set of cases in a lookup table to determine what needs to be rendered in each cube. Take the square in Figure 1.1 with top left point $(0,1)$, this square has only a single point inside the curve and so this matches a rotation of case 1 meaning that one line needs to be rendered in this square. Linear interpolation is then used between the values of the function that produced the red curve at $(0,2)$, $(1,2)$ and $(1,1)$ to find the points of intersection of the line with the edges of the square. Figure 1.1 effectively shows a drawback of the algorithm, that at low resolutions the resulting surface can lose some detail that exists in the true surface, for this project we ensure that the resolution of the algorithm matches that of the dataset which ensures this detail loss is not possible. Extending to three

dimensions requires a new way to segment and traverse the domain as well as a new lookup table.

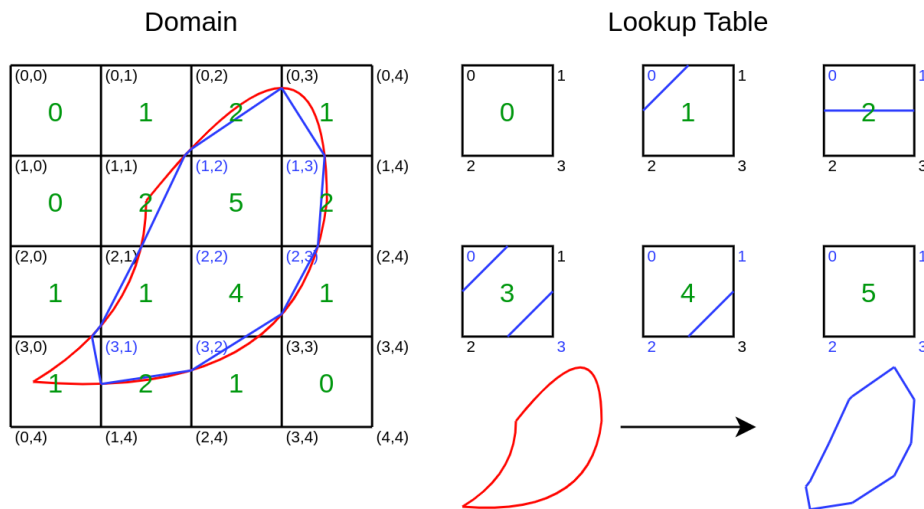


Figure 1.1: Marching cubes example in two dimensions

1.3 Related Research and Alternative Approaches

This project exists in the multivariate visualisation research space. The work of Jankowai and Hotz on defining feature level-sets as previously stated is a direct extension of the work of Carr et al. [6] on fiber surfaces which can be thought of as zero feature level-sets defined on two dimensional traits. Both take the approach of splitting the multi-field datasets into distinct spatial and attribute spaces and describe methods of defining areas of interest in attribute space and displaying the inverse image in the spatial domain.

Lu and Shen [15] approach feature extraction from mutli-variate data in a way that is reminiscent to Carr et al's. but that is applicable to high dimensional attribute spaces. They present a workflow for allowing users to generate their own transfer functions to extract features from multivariate datasets. Their approach is to display to users scatter plots of subsets of attribute space. First they compress the attribute space using weighted kernel density estimation to decrease compute time and then generate a matrix of scatter plots (subspace matrix) which allow users to identify where features based on correlations between pairs of attributes. Users can select a subspace (two dimensional subset of attribute space) and select a feature from this space. The data points covered by this feature are then removed and the subspace matrix is regenerated allowing features that were previously obscured to be seen. This created an iterative procedure to defining what Jankowai and Hotz would call traits, which produces a transfer function that can be used to generate volume renders of the given dataset. The process they describe, though effective, is beyond the scope of a final year project to implement. Where their approach has an advantage over Jankowai and Hotz is that the users do not need to be as familiar with the dataset to produce traits, as they can directly identify areas of interest from subspace matrices.

Blaas et al. [3] choose to address the problem of feature extraction from 3+ dimensional attribute space by performing a projection from the dimension of the attribute space onto a plane, and

then allowing a user to manipulate the parameters of the projection. This has the advantage of being able to visualise the whole of attribute space in a single projection, with the draw back being that it causes a flattening of attribute space, which hides features. However, with the user being able to manipulate the parameters of the projection, they can choose how the data is being flattened. As with Carr et al. the user is able to select a region of attribute space by selecting part of the projection, and see its inverse image in the spatial domain. We feel the primary drawback of Blass et al.'s approach is that its feature selection process using the attribute space projection is quite unintuitive, and would require a user to have an understanding of the projection process to use it effectively.

Buskin et al. [4] offer a unique approach compared to others we have looked at so far, choosing not to let the user interact directly with attribute space when exploring multi-field datasets. Alternatively, they let a user define a point in the spatial domain and then their system identifies points which exhibit similar attributes and renders surfaces around these similar regions. Users can then adjust the how the system decides similarity between points, this is particularly powerful as their system can find trends across all attributes, a task which is very difficult to do when manually inspecting attribute space. This all occurs in real time, and the user can select multiple points and adjust independently the similarity parameters. This approach is particularly useful if the user has an in-depth understanding of their dataset and is interested in a specific spatial area of the dataset and how it may relate to other areas. However, it does not enable users to identify areas of interest in attribute space and explore their inverse images in the spatial domain.

Chapter 2

Methods

2.1 Solution Architecture

A program which allows a user to load multi-field datasets, define traits in that dataset’s attribute space, then generate, render and explore feature level-sets requires six core components:

- **User Interface:** To enable trait definition, level-set customisation, and adjustment of rendering parameters.
- **Dataset Reader:** To read the user’s dataset and provide a standard dataset object that can interface with the rest of the program.
- **3D Environment:** To render feature level-sets meshes and facilitate exploration through a first person camera.
- **Distance Field Generator:** To perform the pullback of the trait into the spatial domain.
- **Marching Cubes:** To take the distance field and generate the level-set meshes.
- **Data Controller:** To manage coordination between all components.

A high-level overview of these components and their interfaces is shown in the component diagram Figure 2.1. The architecture prioritises discrete, loosely coupled components with well defined interfaces. In particular, the dataset reader, marching cubes algorithm, and distance field generator components provide interfaces and maintain no dependencies on other parts of the program. This design enables the independent development and testing of these complex components.

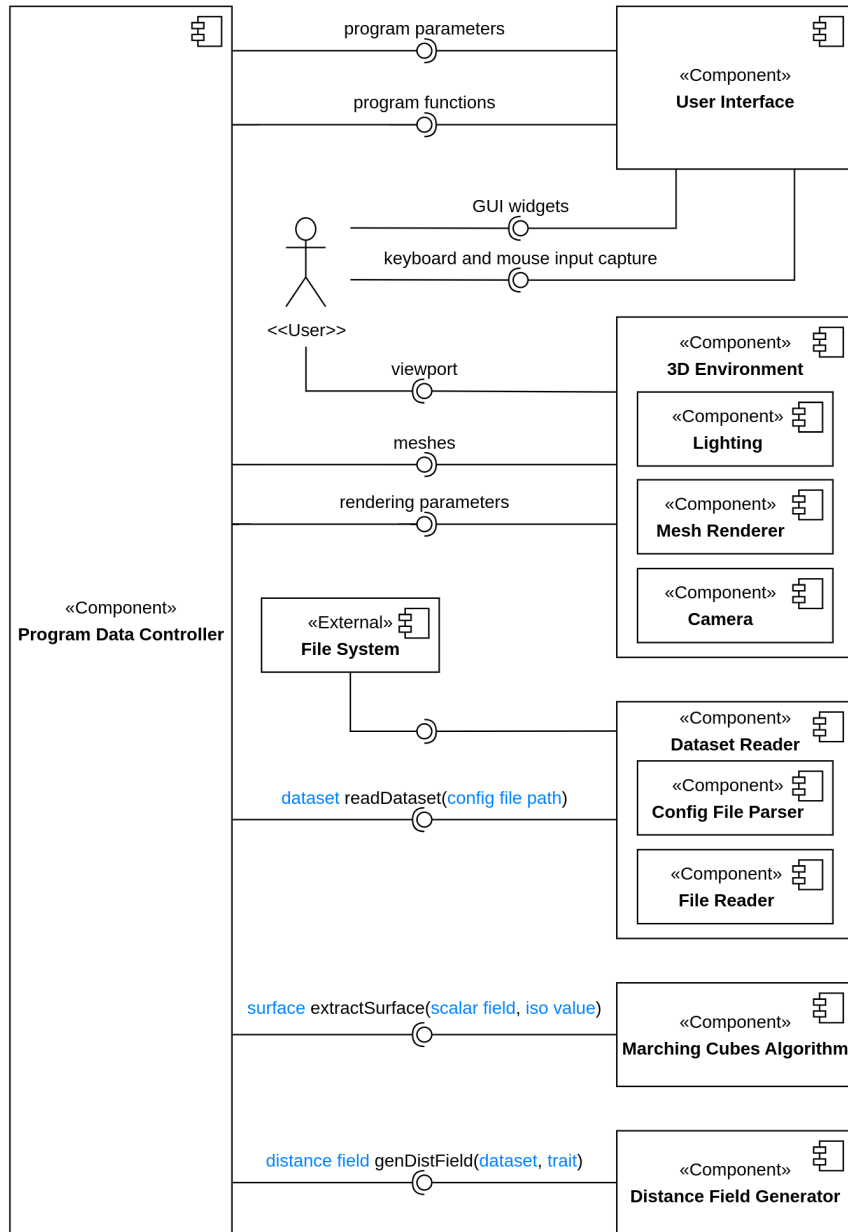


Figure 2.1: Solution component diagram

2.1.1 3D Environment

This project only requires a simple real-time 3D environment in which it is possible to implement mesh rendering, simple lighting and the a first person camera. For the lighting sub component a simplified implementation of the Phong lighting model [19] which omits the original’s specular component while retaining the ambient and diffuse components would suffice. Ambient light is the global light level, diffuse light provides directional lighting and specular light provides highlights and shiny reflections of light on surfaces. Such a simple lighting system is viable as only a single global light source is needed to light, and show the shape of the level-set surfaces. The camera sub component will have to interpret mouse and keyboard inputs and update the projection matrix used to render the scene. The mesh renderer sub component will need to be able to take a set of vertices and normals describing a surface and then render this surface.

2.1.2 Dataset Reader

The dataset reader needs to provide an interface that takes the dataset and produces a standard object which can interface with the rest of the program. Since there is no universal standard for the format of multi-field datasets we developed a configuration file standard. This standard enables users to describe to the program the structure of their dataset. By implementing this configuration file standard users will not have to edit their datasets for compatibility with the program. The configuration file specification is shown in Figure 2.2.

Generic Configuration File Schema <datatype of value, value>

NAME:<string, name of the dataset>
 FILEPATH:<string, global file path of the folder containing the dataset>
 SPACIALDOMAIN:<integer, width>:<integer, height>:<integer, depth>
 DIMENSIONORDER:<string, WIDTH_DEPTH_HEIGHT or WIDTH_HEIGHT_DEPTH>
 INDEXSCHEME:<string, ROWMAJOR or COLUMNMAJOR>
 DATASETSTRUCTURE:<string, ATTRIBUTEPERFILE or BLOCK>:<integer, values per file>
 Then there is a list describing dimensions of attribute space of the form:
 ATTRIBUTE:<string, filename>:<string, attribute name>:<float, min value>:<float, max value>

Figure 2.2: Dataset configuration file schema

2.1.3 User Interface

This solution needs to satisfy two requirements with regards to interfacing with the user. Firstly capturing keyboard and mouse input for controlling the first person camera. Secondly providing widget based interaction through sliders, text-inputs and buttons to allow the user to load datasets, define traits, generate feature level-sets and adjust rendering parameters. There will also need to be a way to manage and or integrate with the viewport provided by the 3D environment.

Figure 2.3 shows a basic design for the graphical user interface which would satisfy the requirements. It has a small collapsible panel in the top right which can handle rendering parameters such as lighting controls and a main controls panel on the left hand side. The controls panel allows the user to load a dataset, define trait points by selecting values from various attributes, and subsequently render, and control the rendering parameters of the feature level-sets.

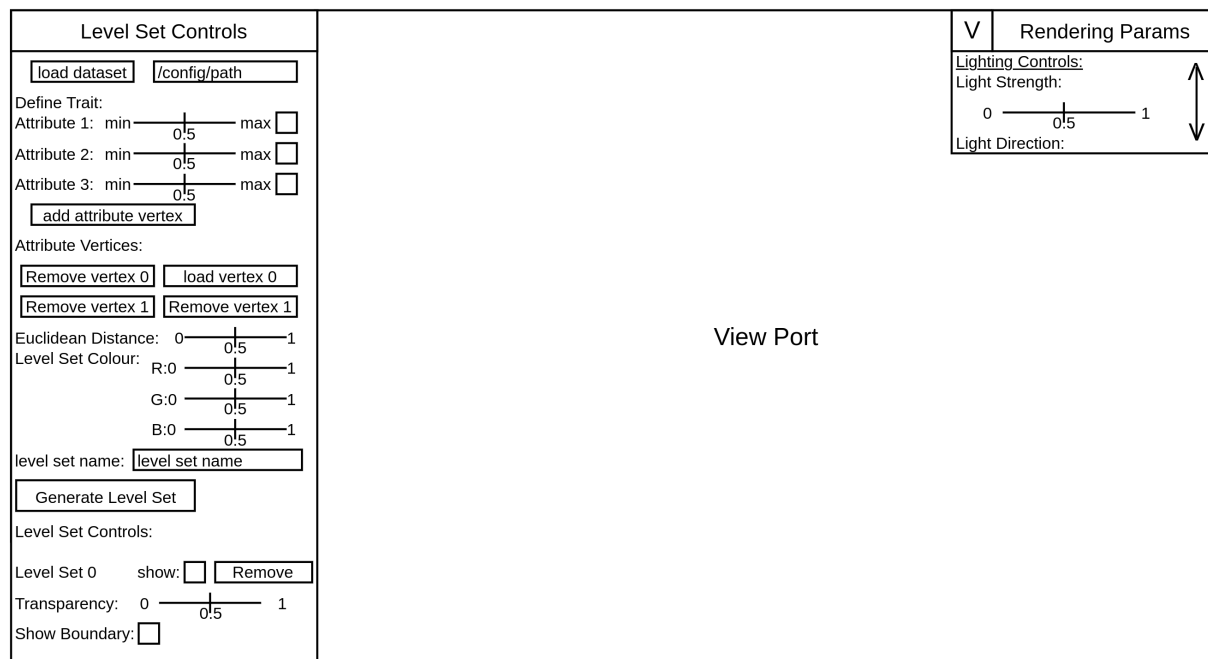


Figure 2.3: User interface initial design

2.1.4 Distance Field Generation

As mentioned in 1.2.5 computing distance fields for traits of arbitrary dimension poses a challenge. Therefore, to simplify the project, we opted to use traits defined explicitly by a set of points, rather than by a set of vertices that describe some geometry. The distance field generator will need to take the trait points specified by the user and the dataset, and then calculate the Euclidean distance from each point in the dataset's image in attribute space to the nearest trait point. It will then return the distance field as a scalar field, a data structure where each point in the domain is assigned a scalar value, in this case, the shortest Euclidean distance from each point's image in attribute space to a trait point.

2.1.5 Marching Cubes Algorithm

The marching cubes component will need to take a scalar field and an iso-value; the resulting surface will be positioned between points with values above and below this iso-value. It will return the surface as a set of vertices and normals describing the triangles of the surface, this can then be passed to the mesh renderer in the 3D environment to render the surface. Recall from section 1.2.4, that feature level-sets are defined as the set of points which share a distance from the trait in attribute space, the iso-value passed to the marching cubes component is the distance from the trait for the target feature level-set being rendered.

2.1.6 Program Data Controller

The existence of a central data controller means that the components can exist stateless without dependencies on other components. In Figure 2.1 the program data controller is seen to be a single component. Conceptually as the scale of the project increases it may be more effective to consider it as a collection of four independent sub-components, a dataset controller, render parameters

controller, distance field controller and level-set controller, this is because fundamentally they handle data and actions that are independent. For example if a feature was developed to save all current render parameters in a file this would be made easier if this was an independent object which could be saved and loaded.

2.2 Pre-Development Decisions

Before implementing the architecture, decisions had to be made on how its requirements would be realised. For example, which coding language and GUI library would be used.

2.2.1 Version control

Git was chosen as the version control system that would be used for this project, primarily due to our experience with it and also because a Git repository was provided to use by the University through GitHub.

2.2.2 Graphics API & Function Loader

Graphics APIs are software interfaces that allow developers to interact with a system's graphics hardware. Graphics APIs provide the essential tools for real-time rendering and so one is necessary to develop the 3D environment. Since we wanted a cross platform solution the only two feasible options for graphics APIs were Vulkan and OpenGL; Vulkan is a more modern, advanced API but requires greater code volume and complexity. We chose to use OpenGL due to the aforementioned simple requirements of the 3D environment and the ease of use and setup for OpenGL when compared with Vulkan.

Graphics APIs such as Vulkan and OpenGL are specifications that define interfaces which hardware and software vendors must support, rather than being libraries which contain implementation code themselves. The actual implementation of these Graphics API functions is handled by graphics driver developers and are specific to each hardware vendor. For OpenGL, a function loader library is essential because it facilitates the use of OpenGL functions which are not statically linked into applications. These libraries generate a source file and at runtime dynamically load supported OpenGL function pointers provided by the system's graphics drivers. These libraries ensure that an application can access the latest OpenGL capabilities available on the target system. We decided to use the GLAD function loader library, there were other options such as GLEW and GL3W, but GLEW uses static loading of all available extensions at runtime resulting in longer initialisation times and GL3W has limited feature support; therefore, we decided GLAD was most appropriate for this project.

2.2.3 Language

Having chosen OpenGL as our graphics API, C++ was the natural choice of language to use for developing the 3D environment component as it is the most common pairing in industry and as such there is a robust ecosystem of tools, libraries and documentation. Furthermore, there are many GUI libraries available in C++, many with direct integration with OpenGL meaning C++ would be the best choice for developing the UI component. C++ is also the ideal choice

for the rest of the components as they all process massive amounts of data in real time, meaning C++'s speed when compared to other languages becomes a significant advantage.

With OpenGL chosen as our graphics API, small amounts of GLSL (OpenGL Shading Language) will be written to make shaders, which are small programs which run directly on graphics hardware. In this project shaders will be used for lighting and rendering operations.

A small amount of Lua will also be used for the project's build script (see 2.2.5).

2.2.4 Testing

Standard software testing approaches are complicated in graphics based projects as a lot of data processing occurs on graphics hardware which often appears as a black box, this results in a lot of testing being direct inspection of produced renders. As aforementioned we have designed the architecture of this project so that as many of its complex components as possible are isolated from the graphics pipeline. We will implement a suite of unit tests for the marching cubes algorithm, dataset reader, and distance field generator components to confirm their reliability independently of the 3D environment, where testing is substantially more challenging.

There are a number of testing libraries available for C++ but considering we only needed the ability for setting up a suite of unit tests, all libraries we considered were suitable. We ultimately chose to move forward with GoogleTest due to its ease of integration and setup.

2.2.5 Build Systems

When creating a complex C++ project with library dependencies and many components it is cumbersome to manually compile the project by invoking the compiler from the command line. Therefore, build systems have been developed, tools which allow you to describe your project at a high level and then either directly compile the project into an executable format, or create platform specific build scripts. Here is a list of build systems which we considered:

- **GNU Make:** A classic tool that requires the user to write a build script which is then ran to compile the project. GNU Make scripts often need to be adapted or rewritten to target different platforms.
- **CMake:** It uses platform agnostic CMake configuration files to generate build scripts such as GNU Make build scripts for a target platform.
- **Premake:** Similar to CMake, but Premake uses Lua scripts rather than configuration files to generate build scripts for a target platform. Premake scripts use a simpler syntax when compared with CMake configuration files.
- **SCons:** It uses python scripts to define build logic and dependencies and uses these to directly compile projects without generating intermediate build scripts.

We opted for Premake as our build system due to its straightforward scripting syntax, although CMake and SCons remain viable alternatives.

2.2.6 User Interface Libraries

We wanted our chosen GUI library to be lightweight and easy to setup as the primary aim of this project is reproducing the feature level-set renders from Jankowai and Hotz [12], not the user interface. These are the options that were considered:

- **Qt**: A comprehensive cross platform framework for building user interfaces. It is designed for complex, feature-rich applications and thus requires a lot of setup to get running including configuration files, custom classes for each window, a large number of library dependencies and even its own compiler, the Meta-Object Compiler. OpenGL content can be embedded in Qt windows.
- **wxWidgets**: Another comprehensive framework like Qt, which requires less setup when compared to Qt but is still designed for complex applications. OpenGL content can be embedded in GUIs using the wxGLCanvas class.
- **Dear ImGui**: Bloat-free, lightweight library particularly suited to integrating into graphics projects. It is immediate-mode, meaning that the UI is redrawn every frame. It is designed for prototype applications and scenarios where the UI is secondary. It is also very quick to setup and develop with.
- **Nuklear**: Similar to Dear ImGui, an immediate-mode library targeted at OpenGL. It is designed to be portable, efficient and easy to integrate. It is also suited for applications where the GUI is not the central component.

Due to both limited development time and lack of complex feature requirements for our GUI we landed on either Dear ImGui or Nuklear, and ultimately chose Dear ImGui. For the requirement of capturing keyboard and mouse inputs we considered the following:

- **SDL**: A framework designed for interfacing with peripherals such as audio, keyboards, mice, gamepads and joysticks, it also comes with advanced multimedia features, and the ability to manage windows and OpenGL contexts.
- **GLFW**: A lightweight utility library designed for creating OpenGL windows and managing OpenGL contexts which also provides simple tools for managing mouse and keyboard inputs.
- **Qt**: Along with its GUI capabilities and its support for managing OpenGL, Qt's input handling features would facilitate everything required for managing the first person camera.

Once again ruling out Qt due to its complexity we were left deciding between GLFW and SDL, we chose to move forward with GLFW as this project would not need many of the extra features packaged with SDL.

2.2.7 Other Libraries

- **GLM**: A header-only library designed for mathematics operations in graphics software and is based on the OpenGL Shading Language (GLSL) specifications. As it is based on GLSL it makes the sending of data between your main program and shaders simple and

it also supports all matrix and vector operations that will be required for the first person camera. Alternatives include Eigen and DirectX Math, however, these alternative libraries are more general purpose and do not have the same focus on compatability with OpenGL and GLSL.

2.2.8 The Dataset

Jankowai and Hotz [12] render feature level-sets for three datasets, and though the aim of the project is to make a tool which works for generic multi-field datasets, using one that Jankowai and Hotz used will provide points of reference for evaluation. Of the three datasets used, Hurricane Isabel stood out as the most appropriate for this project, this is because it is the most accessible both literally and conceptually. It is freely available online as it was provided by the National Center for Atmospheric Research for the IEEE visualisation contest 2004 [11], thanks to its use in the contest there is also a lot of documentation surrounding the dataset. The dataset tracks 13 variables across a three-dimensional space measuring 500 x 500 x 100 units, which corresponds to an approximate 2000km x 2000km x 20km area over the North Atlantic. The data is segmented into 48 time steps, and for each time step there is a separate file for each variable which contains the recorded values at every point within the dataset's grid. This results in 25,000,000 values per file and 325,000,000 values per time step. Only a single time step is needed to generate and render feature level-sets, but multiple will need to be investigated to reproduce the Jankowai and Hotz' renders [12]; Jankowai and Hotz were contacted but could not recall the time step they used.

2.3 Development Methodology

Due to the complexity of the project and scale of the development we concluded an iterative agile approach would be infeasible. Instead we decided to spend short sprints developing minimum viable versions of each component listed in 2.1. These components were to be integrated as they were developed, with the goal of creating a prototype that not only meets the project's aims but also offers substantial potential for future development. Figure 2.4 shows a development dependency graph, this shows which components need to be developed before others, arrows show a conceptual dependency. For example, the marching cubes algorithm depends on the 3D environment, the component itself does not have any literal dependency on the 3D environment component, however, it makes sense to develop the 3D environment first so that you can test the marching cubes algorithm by rendering the meshes it produces. These conceptual dependencies were not clear during planning and were discovered in development.

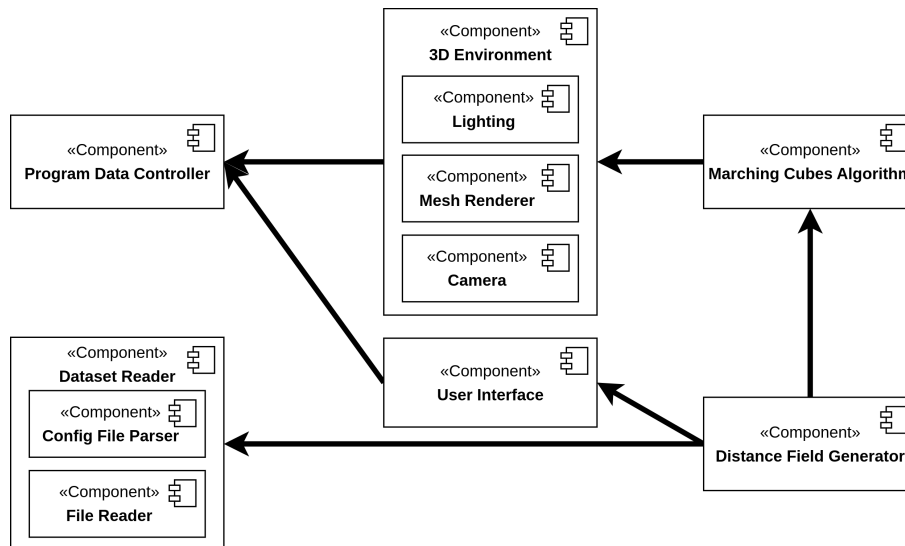


Figure 2.4: development dependency graph

2.3.1 Project Setup

The local Git repository was created and linked with the GitHub repository provided by the university. Figure 2.5 shows the file structure which we decided upon for the organisation of the project's code. The source code is found in the `src` directory with independent code components being found in `src/lib` and the main program being found in `src/main`. The assets directory is reserved for textures, fonts, images and shader programs. The `third_party` directory holds the libraries that the project uses, where a library is small we will download it directly and add it to the repository, however, for larger libraries we will use git submodules to store a read only reference to the libraries repository. Which method does not affect how the code is written or build system is set up. The tests directory will hold unit tests for some complex components of the solution.

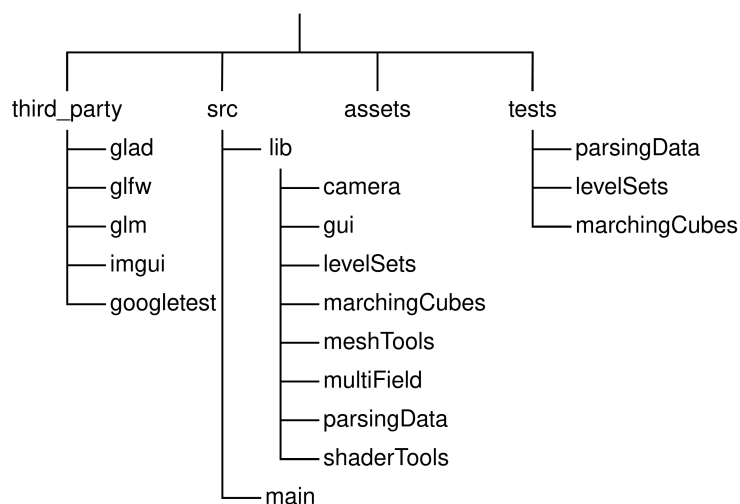


Figure 2.5: Project directory structure

Chapter 3

Implementation

3.1 Implementation

Having discussed in-depth the architecture of a solution and the subsequent pre-development decisions that were made, we move on to our implementation, focusing individually on each component shown in Figure 2.1. Each class mentioned has an accompanying UML diagram in Appendix C, which also states which file the class is defined in, there are also UML diagrams which show the relationships between classes. Unfortunately due to the complexity of the project there are too many UML diagrams for them to be included in the main body of the report. Henceforth class and function names written in the `verbatim` styling, functions will be followed by brackets (), but do not show the expected arguments.

3.1.1 Program Data Controller

As Figure 2.4 suggests, some implementation of the program data controller should be implemented first. Despite 2.1.6's discussion we chose to implement the program data controller as a single class to reduce code complexity, the `ProgramState` class. The class holds most of the rendering parameters for the 3D environment, having them centralised in the `ProgramState` class allows them to be accessed and updated by the user interface. Testing for this component is difficult as it exists as a communication layer to link together the components of our system, therefore there are no unit tests for `ProgramState`.

3.1.2 3D Environment

The 3D environment was implemented directly in the program's main file, with sub-components such as the camera and mesh rendering function being separated into their own files. The initial 3D environment, was set up by loosely following the Getting Started article from Learn OpenGL [20], a `ProgramState` object is created at initialisation and pointed to by GLFW's window user pointer which ensures that the object is available anywhere in the program at runtime. In order to use shader programs the `ShaderProgram` class was created. A `ShaderProgram` object is created with a list of shader source files (which are located in the assets directory) which it subsequently compiles and links. `ShaderProgram` objects can then be used to send data to graphics hardware and to tell OpenGL which shaders it should currently be using.

With the `ShaderProgram` class created, basic shaders to control lighting and rendering could be created and tested. There are two types of shader needed for this project, the vertex shader and the fragment shader. The vertex shader transforms vertex positions from object space (the vertices' position in the 3D environment) to screen space (the vertices' position on screen), see Figure 3.1, while also calculating and passing transformed normal vectors and fragment positions to the fragment shader. The fragment shader then computes the final colour of each pixel (or fragment in OpenGL's terminology), by applying lighting calculations and using the normal

passed from the vertex shader and values passed straight from the main program.

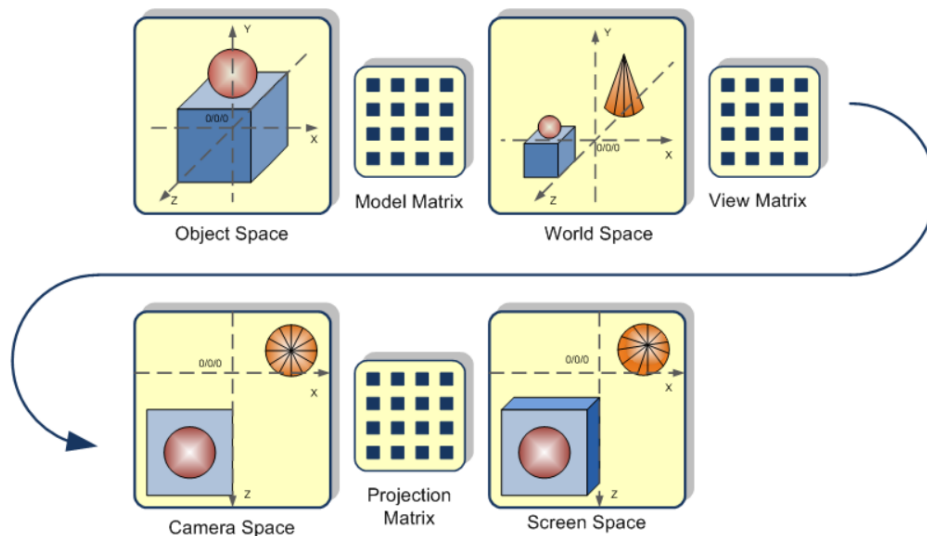


Figure 3.1: How object coordinate space is converted to screen coordinate space using transformation matrices. [13]

As discussed in 2.1.1 we chose the Phong lighting model [19], our implementation is based off Learn OpenGL’s Basic Lighting article [21]. We chose to remove specular lighting calculations from the fragment shader as there are going to be no point lights in the scene. We also chose to add a subtle back light in the opposite direction to diffuse light to ensure surface geometry could be seen even when the surface is facing away from the diffuse light.

We enable mesh rendering using the `createVAO()` function, it takes an array of vertices and normals to those vertices and converts them into a vertex array object, an OpenGL data structure that is used to supply vertex data to the graphics pipeline for rendering. These VAO objects can be thought of as mesh objects as when it comes to rendering, you bind to the VAO and then call an OpenGL draw function to render the mesh in the scene.

The first person camera was implemented through the `Camera` class, a `Camera` object is initialised as part of the `ProgramState` object’s construction, it keeps track of the current position of the camera and where it is looking. The camera’s implementation is based of the Learn OpenGL’s Camera article [22]. Since the camera is intended to be an independent component we deviated from the article and moved key press processing to an independent `processInput()` function and chose for mouse inputs to be captured directly in the render loop. A check is performed in the render loop to see if the UI is being interacted with, this determines whether the updates to user input cause an update to the camera. We chose to not use GLFW callbacks for capturing inputs as they would consume user inputs removing the ability for users to interact with the user interface. Figure 3.2 shows the result of the implementation of the 3D environment, a red cube being rendered, shown from multiple angles by moving the camera.

It is difficult to write tests for the 3D environment as much of the data processing occurs on graphics hardware and the final output is a visually streamed through the viewport. Therefore testing was done through usage, for example the lighting and camera were both tested by rendering a number of cubes like the one in Figure 3.2 and moving the camera around them.

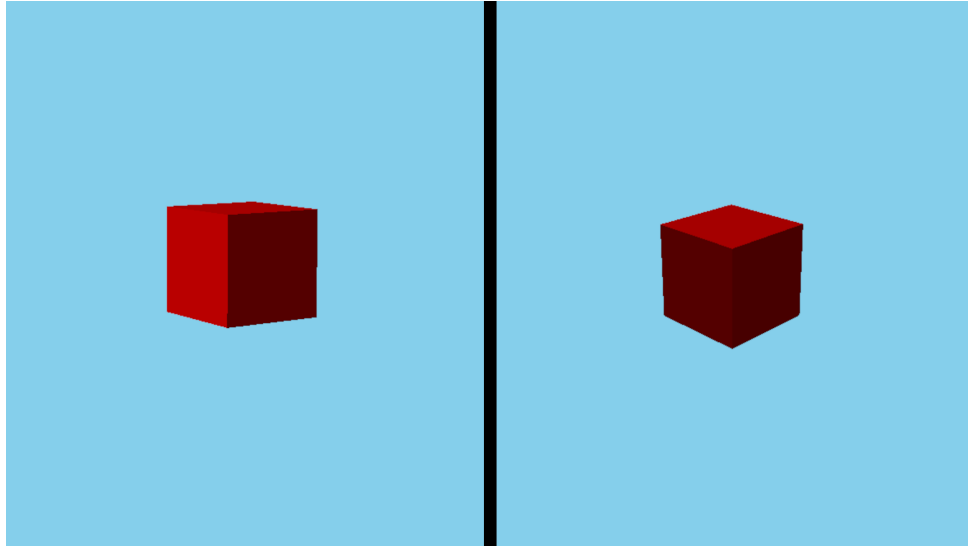


Figure 3.2: Cube rendered after 3D environment implementation.

3.1.3 Dataset Reader

The implementation of the dataset reader program followed the data flow diagram in 3.3, starting first with the file reader then the parser, and finally the dataset reader itself bringing them both together. First we downloaded the Hurricane Isabel dataset from the US National Science Foundation’s National Center for Atmospheric Research [17].

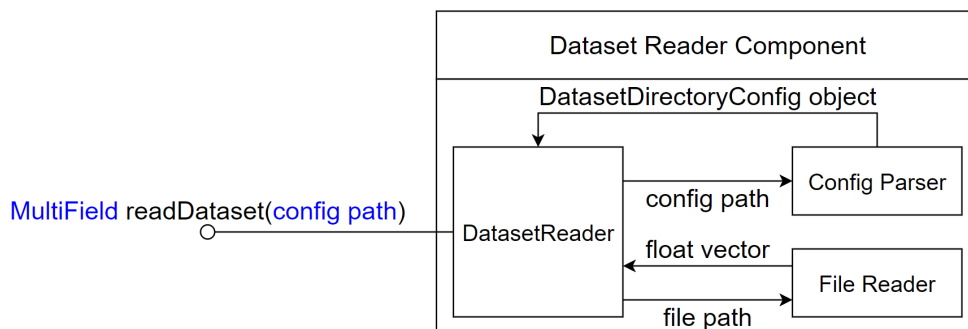


Figure 3.3: Data flow diagram for the dataset reader component.

We started with the file reader sub-component to allow validation that the dataset had been downloaded correctly. The dataset’s files are in the brick of floats format, meaning that the files are a block of sequential floats with no padding or metadata, this made reading them trivial. However, the byte order is big-endian, so after reading the floats into a vector we apply a transformation to the vector to reverse the byte order. The file reader is tested with a unit test which validates the values read from some test binary files. The test binary files are not included in the repository due to their size, but a python tool to generate them is included in with the tests in the repository (binaryFileMaker.py). These unit tests do not test the byte order transformation, however, this is tested in the unit test for the `readDataset()` function which uses a mock dataset which is in stored in reverse byte order.

The parser sub-component takes a file path and reads the file at that location line by line checking that it follows the schema in figure 2.2; if the parser encounters an issue it returns a

`DatasetDirectoryConfig` object with an error flag set to true and outputs an advisory error message to the console. The parser has a set of unit tests which test it against a set of config files which each have a different issue, the unit tests validate that the parser stops and prints out the correct error message.

The `readDataset()` function uses the config parser and file reader to generate and return `MultiField` object, a standard object that is used by the rest of our program to interface with a loaded dataset. If an error is encountered the `MultiField` object will return with an error flag set to true. The `readDataset()` function has a unit test which validates the values of a `MultiField` object it returns when reading a test dataset. The configuration file shown in Figure 3.4 was used to test the `readDataset()` function on the Isabel dataset which are using in this project.

```
NAME:isabel-timestep-02
FILEPATH:/home/michealnestor/University/final-project/dataset/timestep02/
SPACIALDOMAIN:500:100:500
DIMENSIONORDER:WIDTH_DEPTH_HEIGHT
INDEXSCHEME:ROWMAJOR
DATASETSTRUCTURE:ATTRIBUTEPERFILE:25000000
ATTRIBUTE:CLOUDf02.bin:Total cloud:0:0.0032
ATTRIBUTE:Pf02.bin:Pressure:-5471.85791:3225.42578
ATTRIBUTE:PRECIPf02.bin:Precipitation:0:0.01672
ATTRIBUTE:QCLDF02.bin:Cloud moisture mixing ratio:0:0.00332
ATTRIBUTE:QGRAUF02.bin:Graupel mixing ratio:0:0.01638
ATTRIBUTE:QICEf02.bin:Cloud ice mixing ratio:0:0.00099
ATTRIBUTE:QRAINf02.bin:Rain mixing ratio:0:0.01132
ATTRIBUTE:QSNOWf02.bin:Snow mixing ratio:0:0.00135
ATTRIBUTE:QVAPORf02.bin:Water vapor mixing ratio:0:0.02368
ATTRIBUTE:TCf02.bin:Temperature (Celsius):-83.00402:31.51576
ATTRIBUTE:Uf02.bin:X wind speed:-79.47297:85.17703
ATTRIBUTE:Wf02.bin:Y wind speed:-9.06026:28.61434
ATTRIBUTE:Vf02.bin:Z wind speed:-76.03391:82.95293
```

Figure 3.4: Configuration file for time step 2 of the dataset.

3.1.4 Marching Cubes

Our implementation of the marching cubes component follows Lorsen and Cline’s original description of the algorithm [14], the case lookup and edge tables were sourced from the project supervisor Hamish Carr, though they are widely available online. Our implementation is accessible through the `extractSurface()` function which expects a two-dimensional float vector, with each inner vector storing four floats, the three coordinates of the point and its iso-value. This vector is the same format as the distance field returned by the distance field generator. The vector is then looped over, the triangles of the surface are identified, their surface normals are calculated and are stored with the coordinates of the vertices surrounding the triangles in a `Surface` object and that `Surface` object is returned. The `extractSurface()` function has a set of unit tests which test small surfaces and with known vertices such that the output can be validated, and a test which uses the case tables to generate each of the marching cubes cases and then validate that the function renders them correctly.

The `Surface` object has a method `getVertices()` which returns a float vector containing the surface’s vertices and normals which can then be passed to `createVAO()`, the VAO from which enables the rendering of the surface. This `getVertices()` function is the implementation of the mesh generator sub component in Figure 2.1. The vertices close to the surface are stored so

that they can be rendered by the user, this is useful if they want to see which side of a surface is inside or outside. Figure 3.5 shows a simple surface generated using our marching cubes implementation.

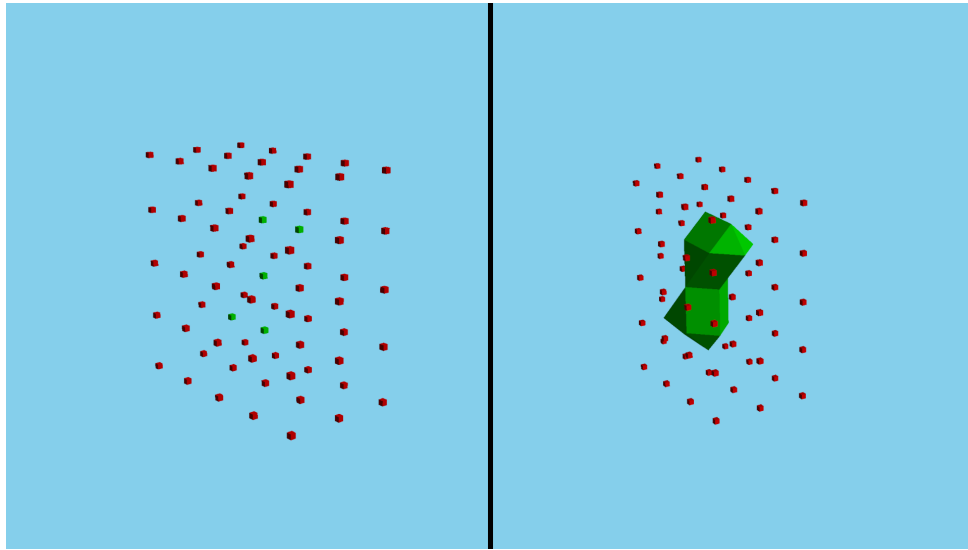


Figure 3.5: Left: Vertices rendered with green set as inside the surface and red outside. Right: Surface rendered on top using marching cubes

3.1.5 Graphical User Interface

Having chosen Dear ImGui, implementation of the GUI was very quick. Dear ImGui works by allowing developers to call functions which generate UI components, these are called every frame resulting in the UI updating each frame. To realise the design in Figure 2.3 we made a simple `Gui` class which stored some local state about UI elements as well as a pointer to the program state object, and defined two functions on it designed to be called in the main loop each frame, one to generate the main control panel and one to generate the rendering parameters panel. The control panel renders programmatically, starting by only showing the load dataset button and accompanying text entry, once loaded it shows the trait controls, and finally once a level set has been rendered it shows the level set controls at the bottom. Figure 3.6 shows the result of the UI implementation. There is no clean way to test ImGui UI's, one would have to either take and analyse screenshots or develop a bot to interact with the UI, so it was tested through usage.

3.1.6 Distance Field Generation

The distance field component was implemented as the `generateDistanceField()` function, it takes a `MultiField` object and a vector containing `TraitPoint` objects which describe the trait. It works by looping through each point in the dataset, then calculating and storing the minimum distance to any of the points which define the trait. There are a set of unit tests that use mock `MultiField` objects and traits to generate a distance field which is then validated against a hand computed distance field. The distance fields are generated using traits defined in one, two and three dimensional attribute spaces, with these tests passing and the maths for further dimensions being identical, we are confident these tests are robust.

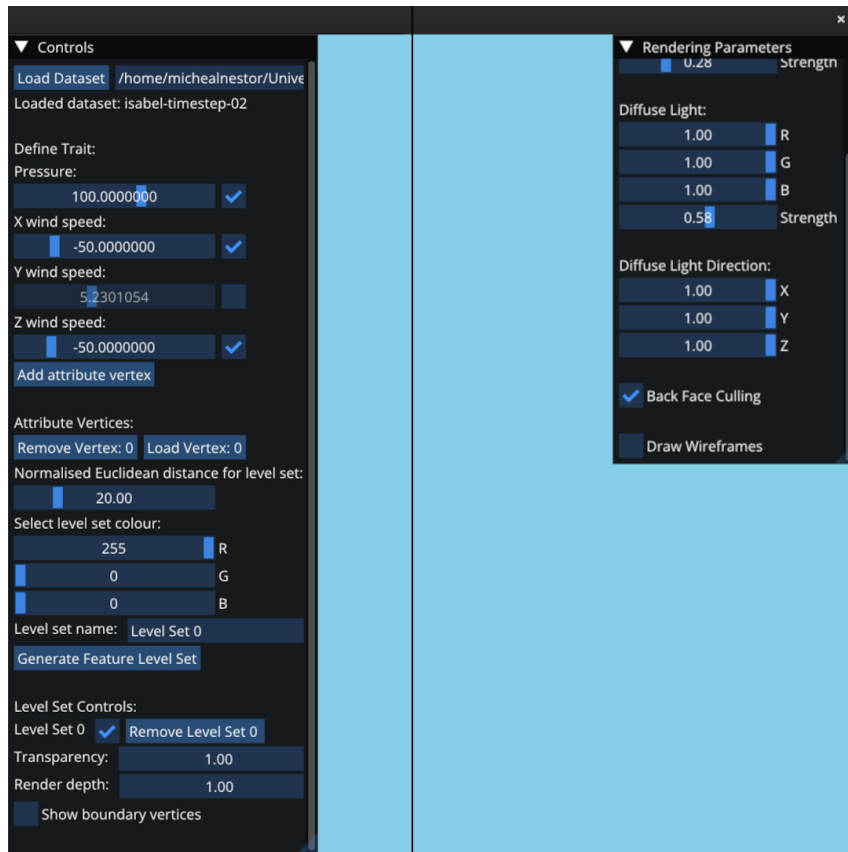


Figure 3.6: GUI implementation shown on the edges of the window

3.1.7 Surface Rendering

When the user selects to generate a feature level-set, a `FeatureLevelSet` object is initialised and stored in the `ProgramState`'s level-sets vector. `FeatureLevelSet` objects require the loaded dataset's `MultiField` object, the trait, defined as `TraitPoint` vector and the normalised euclidean distance for the feature level-set. At initialisation `FeatureLevelSet` objects normalise the trait points relative to their attribute's bounds, generate a distance field and then generate and store a surface along with its VAO using the `createVAO()` function. The normalisation step occurs as the user defines the trait in attribute space, but for distance field generation, all attributes are normalised to alleviate difference in attribute value scales. `FeatureLevelSet` objects also store rendering parameters for the surface such as colour, opacity, whether to show the boundary vertices surrounding the surface and a render depth which is used by the render loop to decide which surfaces are rendered first. This has no effect on opaque surfaces but matters when transparency is enabled. Since `FeatureLevelSet` are accessible in the program state, the main loop can access the level-set's VAOs and rendering parameters and render them using the OpenGL's draw functions. Furthermore, the GUI can also access them, which means the user can update the rendering parameters in real time. Testing of this process involved directly attempting to render feature level-sets from the Isabel dataset.

Chapter 4

Results and Discussion

4.1 Rendering Feature Level-Sets

Our solution succeeds in its aim to allow users to describe and load their multi-field datasets, define traits in their dataset’s attribute space and subsequently generate and explore renders of feature level-sets. We will demonstrate its success by rendering some feature level-sets.

Identifying where feature level-sets occur requires a detailed knowledge of the dataset as it requires the user to understand how the attributes are distributed. To gain this understanding we generated a histogram for each of the attributes of the dataset, these histograms allowed us to see where features might exist. The code used to generate these histograms, is not included in the repository but can be found in Appendix D. Figure 4.1 shows us that for time step 5 there is a relatively even distribution of temperatures across the range of values available (TCf05.bin), however, for water vapour mixing ratio (QVAPORf05.bin), values are collected close to the minimum. This is significant as depending on the Euclidean distance chosen, feature level-sets may encompass a small subset of total data points for some attributes, but include a majority of the dataset’s points for others effectively making the inclusion of the latter attributes in the trait irrelevant. This issue is visualised in 4.2, where the inclusion of the water vapour mixing ratio (QVAPORf05.bin) is likely irrelevant as it is within 5% of the value range of the point chosen for this attribute. This highlights an issue with using a global Euclidean for all attributes which will be explored later in this chapter.

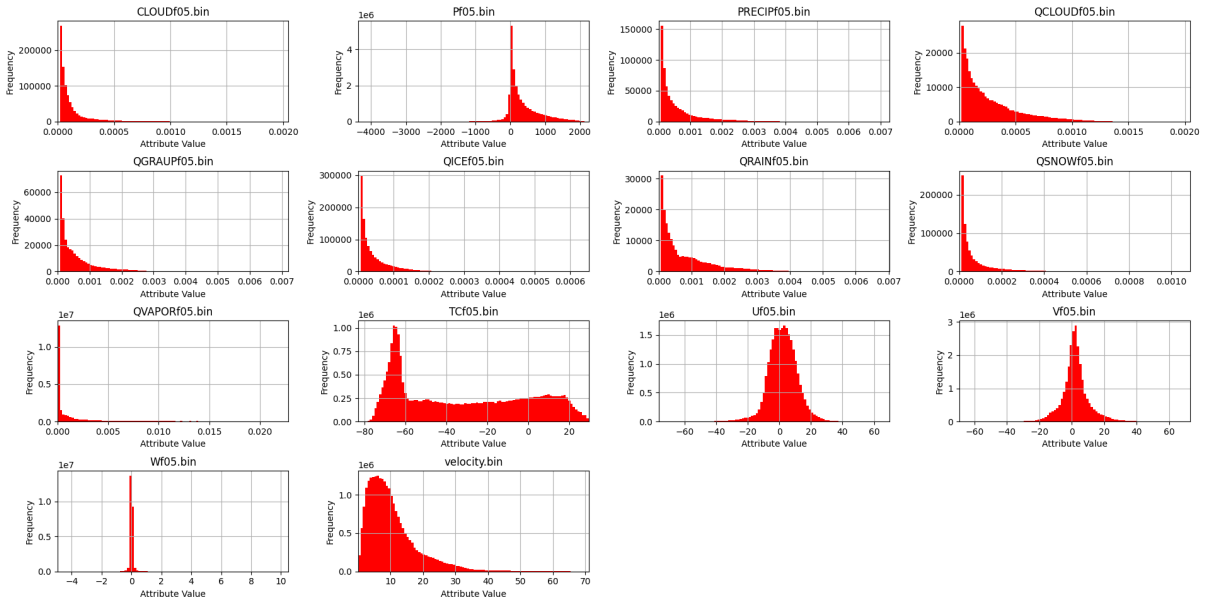


Figure 4.1: Attribute histograms for time step 5 of the Isabel dataset

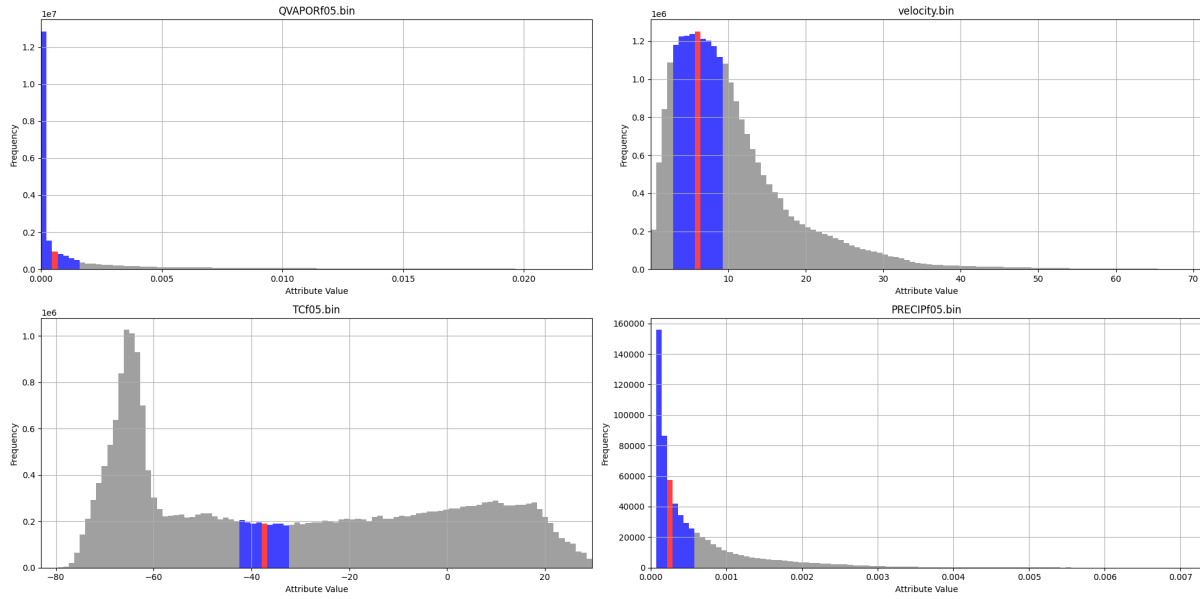


Figure 4.2: Histograms showing what proportion of the dataset’s data points would be encompassed by feature level-set with a single point trait defined in four attributes with a Euclidean distance of 5.

The histograms in Figure 4.1 show there is a relatively wide distribution of values for the temperature (TCf05.bin), cloud moisture mixing ratio (QCLOUDf05.bin) and velocity attributes of time step 5, this means that a shared Euclidean distance would not drown out one of the attributes and so they are ideal to include in an attribute space for a feature level-set. It is useful to render a set of iso-surfaces for each attribute as a feature level-set can be thought of as the region overlap of iso-surfaces. Figure 4.3 shows this process, it shows an iso-surface for each attribute and the iso-values which generated them, it also shows the resulting feature level-set surface when you combine each iso-surface’s iso-value into a single trait point.

4.2 Measuring Success

4.2.1 Efficacy

Measuring success on this project is difficult, the independent components have been validated with test scripts. However, the distance field component tests compare program output to hand computed distance fields, meaning naturally we are only testing against using relatively simple traits. Validating the final renders using full distance fields is more challenging. Ideally, we would validate by passing the same parameters used to generate our feature level-sets into the software created by Jankowai and Hotz and compare the results, but we use different trait definition systems and different rendering methods, so identical renders would not be possible, and even if they were, with our current implementation being un-optimised we do not have the computational power required to automate such a test with the time available.

Since our feature level-set renders do look as we would expect, we decided to validate our efficacy by attempting to render the same feature level-sets that Jankowai and Hotz show in their paper

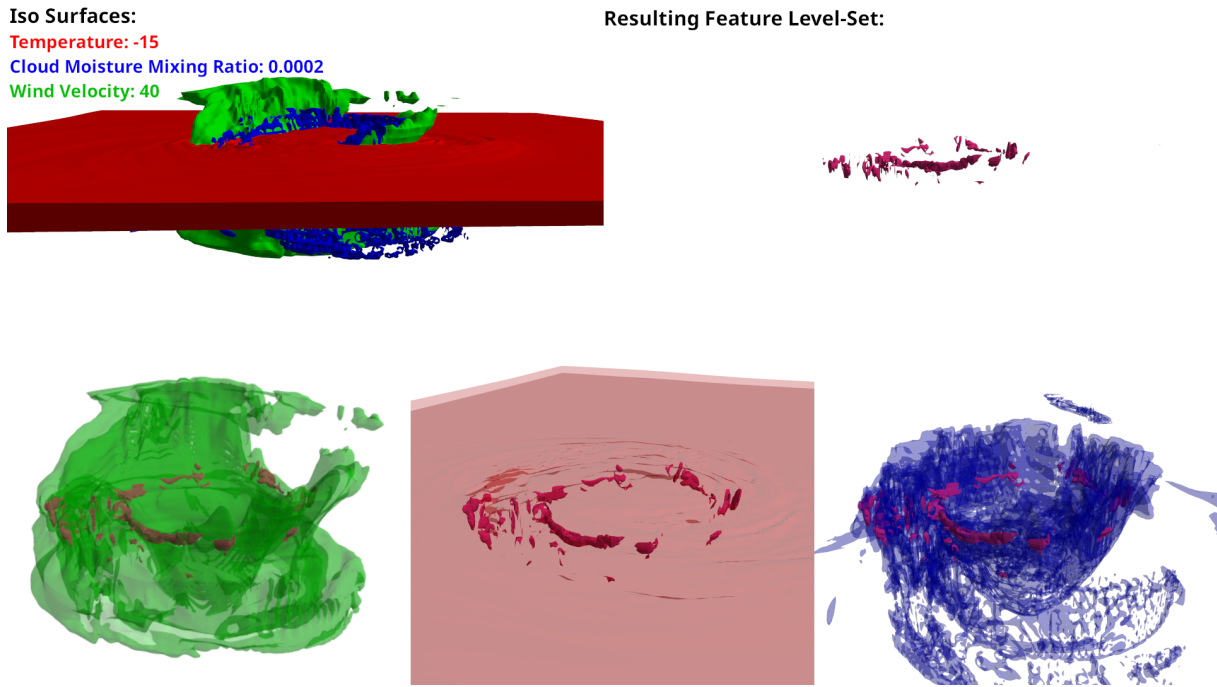


Figure 4.3: Three iso-surfaces and the feature level-set which is their over lap. Rendered using data from time step 5 of the Isabel dataset, with a normalised Euclidean distance of 5, translating to values $\pm 5\%$ of the value range falling inside each surface.

[12], see Figure 4.6. In their paper, Jankowai and Hotz do not state clearly the trait used to generate, and when contacted can not recall the time step of the dataset that they used. They do include a diagram of the parallel coordinate axis they used to define the trait, Figure 4.5, which clearly shows they used combination of precipitation and velocity for their attribute space, but does not show the actual values used to define the trait. Velocity is not an attribute of the Isabel dataset, when contacted Hotz explained that the velocity was the "magnitude of velocity in every point, a derived quantity", we created a python script to generate this derived quantity which is included in section C.0.1.

Though Jankowai and Hotz use a different trait definition method to us, from Figure 4.5 it seems reasonable that a trait consisting of a 0 velocity and 0 precipitation paired with a large Euclidean distance value would yield similar feature level-sets. Inspection of Figure 4.6 shows the outline of Florida and the Bahamas on the top right of the render, using Bell's best track diagram of the Hurricane [1] annotated to show the dataset's region and start point, Figure 4.4, along with the relative position of the eye of the storm to Florida in Jankowai and Hotz' render, we can tell that they used an early time step. Therefore, we generated feature level-sets with the aforementioned trait and Euclidean distance for each of the first six time steps of the dataset, and compared them to Jankowai and Hotz looking for similarities. We found time step 5 to be the most similar, and using this time step we were able to generate the feature level-sets shown in Figure 4.7, though not identical, on close inspection they are very similar, and so we conclude our aim of reproducing Jankowai and Hotz' work a success.

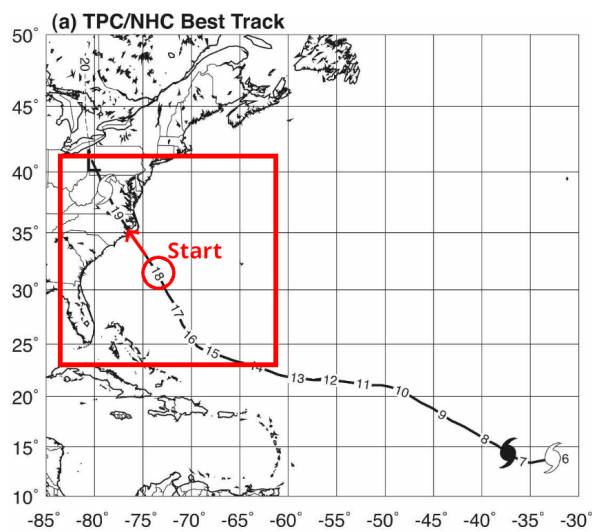


Figure 4.4: Bell's Isabel track diagram [1] with a red rectangle showing the region covered by the Isabel dataset, and the start point of the dataset labelled.

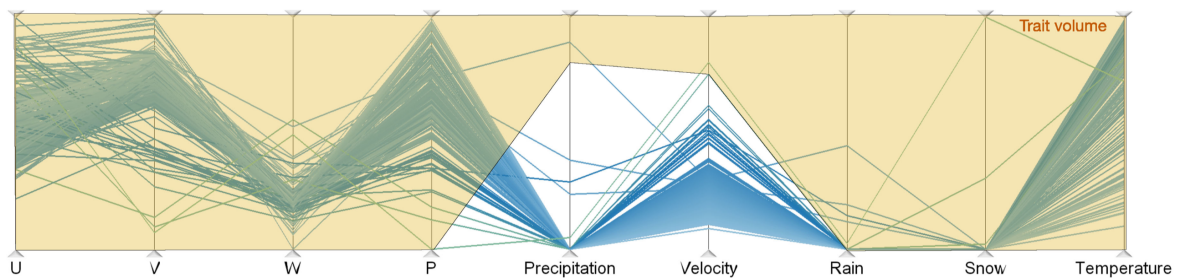


Figure 4.5: Jankowai and Hotz' Isabel trait definition [12]

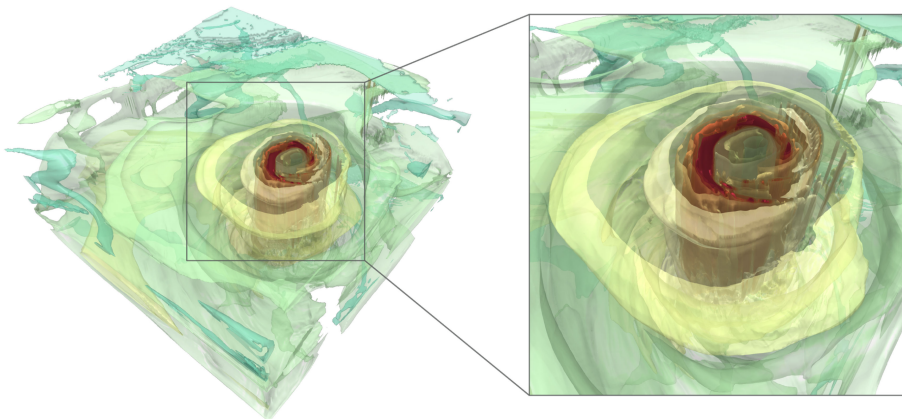


Figure 4.6: Jankowai and Hotz' Isabel feature level-sets [12]

Dataset: Hurricane Isabel - Timestep 5

Trait: Precipitation = 0, Wind Velocity = 0.0018630

Normalised Euclidean Distances For Each Feature Level-Set: 10, 30, 50, 57.5, 62.5, 85

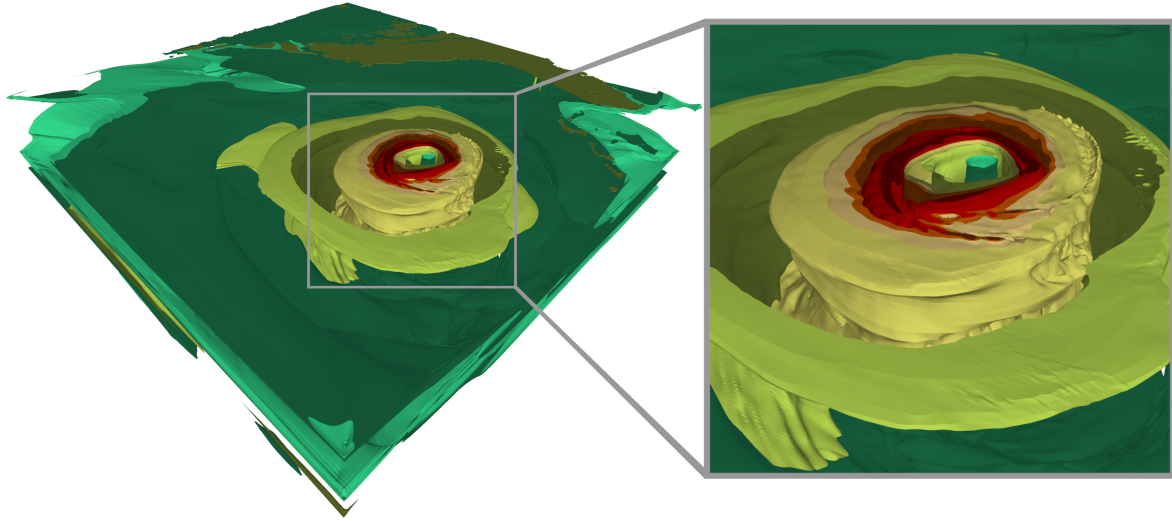


Figure 4.7: Our reproduction of Jankowai and Hotz' feature level-sets

4.2.2 Performance

With this project being a prototype we focused on feature development over optimisation, this means there is a lot of room to make the program run faster. With that said it is still important to measure the performance we are seeing with the current version of the program. All performance measurements were ran on a computer running the Fedora 39 Workstation operating system, with a AMD Ryzen 7 5800X CPU, 32GB of DDR4 3600 MT/s dual channel memory and an NVIDIA GeForce RTX 4070 Ti GPU. Figure 4.8 shows our measurements, the dataset reader was measured using different sizes of dataset, the distance field generator was measured with different amounts of identically sized attribute dimensions included in the trait and the marching cubes was measured with different sized surfaces. All show a linear relationship with respect to time. Considering the lack of optimisation of the program, the results are very impressive with a worst case scenario of a feature level-set with a 14 dimensional trait taking less than 10 seconds to generate. We also measured frame rate, but could not get it to fall below 2000 frames per second when uncapped, and since we chose to cap the program at 240 frames per second we conclude that the overall performance of the program is acceptable for a prototype and ready for usage. These impressive results do not excuse the need for further optimisation, as this is a powerful computer and no tests have been run on more typical consumer grade computers.

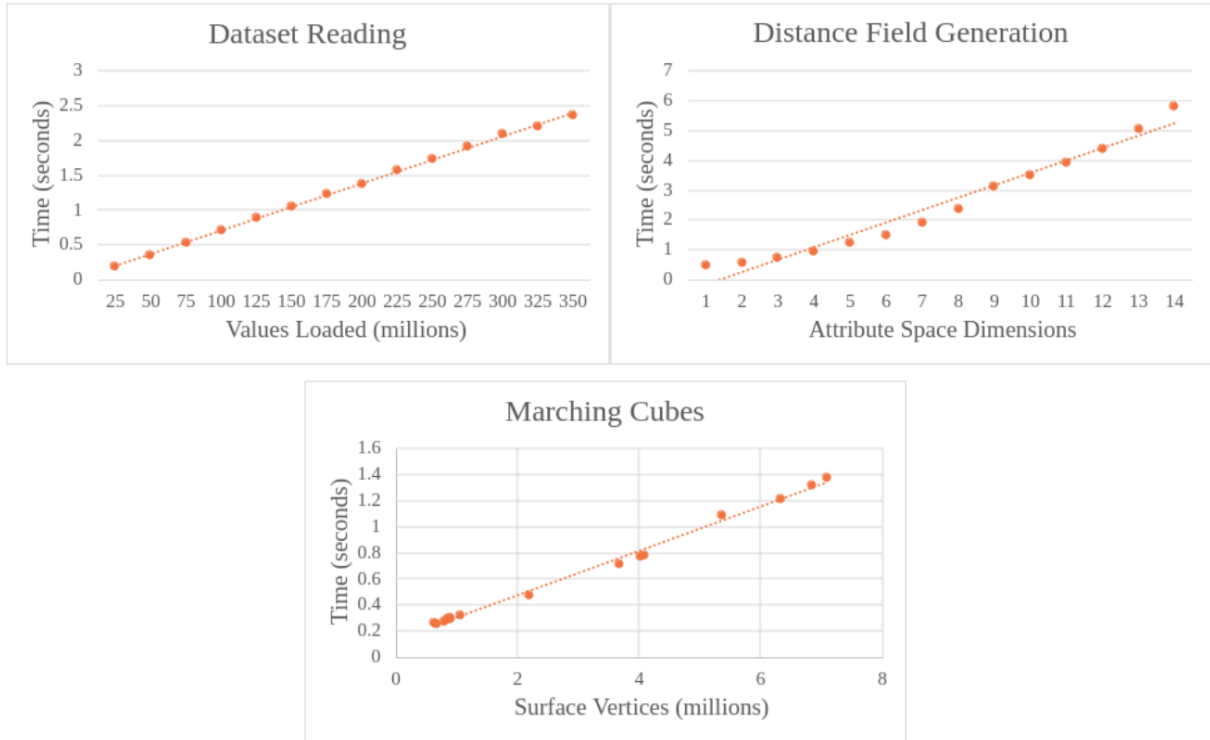


Figure 4.8: Performance measurements for our program.

4.3 Future Development

Having discussed the success of this project, we will now discuss areas for future work and development on the project.

4.3.1 Separating Distance Fields from Feature Level-Sets

In our current implementation feature level-sets and distance fields are a single object, the user defines a trait, selects a Euclidean distance, then generates a feature level-set. This process means that every time the user wants to generate a new feature level-set without changing the trait, a new but identical distance field is generated and stored. Therefore, we propose that the distance fields and feature level-sets should be decoupled. A better system would allow users to create and manage a set of distance fields tied to traits, and then separately generate feature level-sets based on which trait is currently selected.

4.3.2 3D Environment

The current 3D environment implementation is suitable but basic, and there are some features which would enhance it:

- **Order-independent transparency (OIT):** Currently, the user can choose the order in which surfaces are rendered and surfaces are rendered one at a time. When opaque this does not cause a problem, however, when surfaces are made transparent the surface rendered first will appear entirely behind the surface rendered after, which poses a problem if they intersect. OIT is a technique which ensures transparent objects are rendered correctly

without artifacts irrespective of render order. Jankowai and Hotz [12] suggest using the A-buffers OIT technique introduced by Carpenter [5] and point to the work of Maule et al. [16] for a modern efficient approach. We were unable to implement OIT within the allotted time for the project.

- **Spatial Awareness:** Currently, it can be difficult to understand where you are in space when exploring a render. It would be useful if a boundary showed the edges of the dataset. Also for spatial datasets like Isabel if there was a conversion between the coordinate space of the render and the coordinate space of the dataset to inform the user of where they are.
- **Animation:** For temporal datasets such as Isabel, it may be useful for researchers to be able to see how a feature level-set develops over time. With the current system they are able to load each time step and render the same feature level-set, a feature could be developed that would allow them to animate the transition of level-set's between time steps.

4.3.3 Distance Field Generation

Distance field generation is a trivially parallelisable problem, as it is a simple mapping with no dependencies between calculations, however, our current implementation is serial. A simple improvement would be to utilise multi-threading to spread the work over multiple CPU threads but a better solution would be to move the calculations onto the GPU which would offer a massive performance increase given the large thread counts and relatively simple calculations required for calculating each point's distance to the trait.

As discussed in 1.2.5, we only allow for traits to be defined explicitly as a set of points, and do not allow for geometric traits. A goal for future development should be to enable the definition of complex geometric traits and to accurately calculate the distance field surrounding them. This development should occur after distance field generation has been moved to the GPU as calculating the the shortest distance from a point to a geometric trait is a lot harder computationally then calculating the shortest distance from a point to a set of point traits.

4.3.4 Marching Cubes

Our current implementation of marching cubes uses the basic algorithm described by Lorensen and Cline [14] without any of the efficiency enhancements mentioned in their original paper or any subsequent ones. We chose not to spend time enhancing our implementation as can be seen from our performance analysis in 4.2.2 the algorithm performed efficiently enough for our current needs. A goal for future development should be to implement the efficiency improvements and to parallelise the implementation using either multi-threading or ideally moving it onto the GPU, as with distance field generation, marching cubes is a trivially parallelisable problem.

Our current implementation does not include smooth shading of the surface, each vertex of each triangle has as its normal, the face normal of the its triangle. This means that there are not smooth lighting transitions between surfaces the triangles of the surface, this can be distracting for a user and can also make comparison with renders produced by other programs. As we already have the face normals for each triangle, smooth vertex normals can be calculated using

an average of the normals of the faces the vertex is connected to. This would add overhead to the algorithm and we did not have a chance within the allotted time to implement it.

4.3.5 Dataset Reader

The current dataset reader performs well, however, this performance could be increased if multi-threading was utilised to read files of the dataset in parallel. Furthermore, there is room for improvement in our current configuration file standard as not all multi-field datasets can be described, and the reader currently only supports a single style of dataset (attribute-per-file datasets). The program should support more ways of describing and organising multi-field datasets.

4.3.6 User Interface

Though we stand by our assessment in 2.2.6 that Dear ImGui was the correct choice for the rapid prototype development of this project, we feel that we would not have been able to develop the current UI feature set if we had used a more complex library. However, it has become clear that Dear ImGui is not the most effective GUI option for this program. Dear ImGui's is not designed for complex GUI interaction and struggles to provide effective tools for defining traits and managing multiple feature level-sets. We believe that moving forward a more advanced UI system built in a library such as QT would be more suitable for this project. This would allow for a cleaner user interface where specific trait and level-set controls could be moved to pop-out UI elements and the UI itself would not need to be drawn over the viewport, rather the viewport could be embedded in the panel of the UI. This would also enable interactions such as saving feature level-sets, distance fields, traits and rendering parameters to files to be added to the UI without cluttering it further, these could exist in a separate panel.

4.4 Future Research

Finally, we will address areas that we believe still require further research.

4.4.1 Measuring Euclidean Distance to a Trait

As displayed in Figure 4.2, using a global Euclidean distance can drown out traits where data values are bunched together. This can be resolved by defining multiple trait points which share a value in the bunched attribute and span a range of values in the non bunched attribute. For example, if the range of attribute values is 1-100, a trait is selected as a single point at 5, and the Euclidean distance is set at 2, points that map to values 3-7 will be inside the surface, this is the same as two trait points being selected at 4 and 6 with Euclidean distance of 1. This could also be resolved by not having a global Euclidean distance, and instead having a local Euclidean distance to each attribute, which would involve a complication of distance field generation but may be more intuitive for a user. We feel that research into the best approaches to overcoming the drawbacks of using a global Euclidean distance to a trait would be valuable to ensure we are offering the most value to the users of the software.

4.4.2 Euclidean Distance as a Metric

While working on this project we also considered the effectiveness of using Euclidean distance as a metric when measuring difference between two points in attribute space. Consider Figure 4.9, if the horizontal and vertical axis represent physical measures, for example meters, the distance 5 is meaningful, it shows a distance in space of 5 meters. However, if the horizontal and vertical axis represent measures of two attributes, the meaningfulness of Euclidean distance is less convincing. Typically x units moved in some space has a consistent meaning, but in attribute space x units moved can mean dramatically difference things, for example, if Figure 4.9's horizontal axis represented temperature, and vertical axis represented air pressure, then a distance of x could mean a change in temperature of x and no change in pressure, similar for pressure, or some change in either resulting in a Euclidean distance measure of x . We feel it would be valuable to pursue research into the effectiveness other distance metrics such as Minkowski and Manhattan metrics [7] for distance field generation.

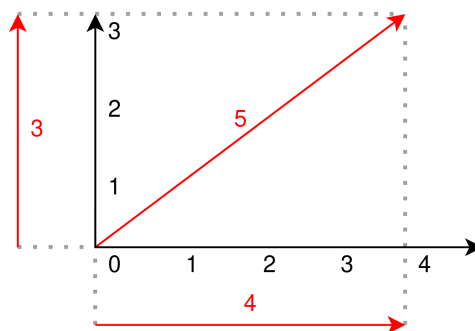


Figure 4.9: Euclidean distance diagram.

References

- [1] M. M. Bell and M. T. Montgomery. Observed structure, evolution, and potential intensity of category 5 hurricane isabel (2003) from 12 to 14 september. *Monthly Weather Review*, 136, 2008.
- [2] M. d. Berg. *Computational geometry : algorithms and applications*. Springer, Berlin, third edition. edition, 2010 - 2010.
- [3] C. P. P. F. H. Blaas, Jorik and Botha. Interactive Visualization of Multi-Field Medical Data Using Linked Physical and Feature-Space Views. In K. Museth, T. Moeller, and A. Ynnerman, editors, *Eurographics/ IEEE-VGTC Symposium on Visualization*. The Eurographics Association, 2007.
- [4] S. Busking, C. P. Botha, and F. H. Post. Example-based interactive illustration of multi-field datasets. volume 34, 2010.
- [5] L. Carpenter. A-buffer, an antialiased hidden surface method. *Computer Graphics (ACM)*, 18, 1984.
- [6] H. Carr, Z. Geng, J. Tierny, A. Chattopadhyay, and A. Knoll. Fiber surfaces: Generalizing isosurfaces to bivariate data. *Computer Graphics Forum*, 34, 2015.
- [7] M. M. Deza and E. Deza. *Encyclopedia of distances*. 2009.
- [8] R. A. Drebin, L. Carpenter, and P. Hanrahan. Volume rendering. *SIGGRAPH Comput. Graph.*, 22(4):65–74, jun 1988.
- [9] H. Edelsbrunner. *Algorithms in combinatorial geometry*. EATCS monographs on theoretical computer science ; v. 10. Springer-Verlag, Berlin ;, 1987.
- [10] I. Hotz and R. Peikert. Definition of a multifield. *Mathematics and Visualization*, 37, 2014.
- [11] IEEE. *IEEE Visualization 2004 Contest*, 2004. [Online]. [Accessed: 14 April 2024]. Available from: <http://vis.computer.org/vis2004contest/data.html>.
- [12] J. Jankowai and I. Hotz. Feature level-sets: Generalizing iso-surfaces to multi-variate data. *IEEE Transactions on Visualization and Computer Graphics*, 26, 2020.
- [13] I. Kholodov. 19. *Coordinate spaces, cont.*, 2007. [Online]. [Accessed: 14 April 2024]. Available from: <https://shorturl.at/gjkgzQ>.
- [14] W. E. Lorensen and H. E. Cline. Marching cubes: A high resolution 3d surface construction algorithm. *SIGGRAPH Comput. Graph.*, 21(4):163–169, aug 1987.
- [15] K. Lu and H. W. Shen. Multivariate volumetric data analysis and visualization through bottom-up subspace exploration. 2017.

- [16] M. Maule, J. L. Comba, R. Torchelsen, and R. Bastos. Memory-efficient order-independent transparency with dynamic fragment buffer. 2012.
- [17] NCAR. *Hurricane Isabel WRF Model Data Visualization*, 2016. [Online]. [Accessed: 15 April 2024]. Available from: <https://www.earthsystemgrid.org/dataset/isabeldata.html>.
- [18] NOAA. *Hurricane Isabel September 18-19 2003*, 2004. [Online]. [Accessed: 7 May 2024]. Available from: <https://www.weather.gov/media/publications/assessments/isabel.pdf>.
- [19] B. T. Phong. Illumination for computer generated pictures. *Communications of the ACM*, 18, 1975.
- [20] J. D. Vries. *Basic Lighting*, 2014. [Online]. [Accessed: 14 April 2024]. Available from: <https://learnopengl.com/Getting-started/OpenGL>.
- [21] J. D. Vries. *Getting Started with OpenGL*, 2014. [Online]. [Accessed: 14 April 2024]. Available from: <https://learnopengl.com/Lighting/Basic-Lighting>.
- [22] J. D. Vries. *Camera*, 2014. [Online]. [Accessed: 14 April 2024]. Available from: <https://learnopengl.com/Getting-started/Camera>.
- [23] R. Wenger. *Isosurfaces: Geometry, topology, and algorithms*. 2013.

Appendix A

Self-appraisal

A.1 Critical Self-Evaluation

Discussion of features that I feel are missing from the final project can be found in in 4.3. Though I am very pleased with the result of the project, the process was definitely flawed. The biggest problem was getting tunnel vision with coding before effectively planning. The plan which was decided on at the start was very high level, I understood which core components needed to exist but deferred thinking about how the components would integrate until it was actually time to integrate the components together. There is evidence of this in that each feature level-set object stores its own distance field, although in retrospect it makes more sense for these to be separate components where different feature level-sets can reference same distance field. More evidence of this rushed approach to planning can be found in the absence of implementation of order-independent transparency. During preliminary research I glossed over Jankowai and Hotz' comments on A-buffers and only recognised that there was a problem when it came to implementing transparency. The current solution is suitable for surfaces which do not intersect, however, a more in-depth plan into the requirements of the 3D environment could have lead to OIT being able to be implemented within the allotted time. The key lesson I have taken from this is that development should not start on a project until the project is adequately understood. I began development early and learned as the project progressed under the false assumption that I did not have time to produce an adequate plan, however, if I had spent time gaining an in-depth understanding of the architecture required for the project before beginning development, development would have taken less time and the final product would have a cleaner code base.

I found writing the report particularly challenging, having been diagnosed with dyslexia just over a year ago I am still learning the appropriate strategies for projects which require a lot of reading and writing. During this process I let old habits creep in and neglected the report writing until time pressure forced me to begin, this is because the prospect of writing the report was so overwhelming. I am still proud of the report, however, if I had created a strategy for how I would write it, and had written it alongside development it would have resulted in far less pressure and stress as the deadline approached.

Though I am happy with the test coverage of the final code base, tests were not developed as the code was written but added later as a validation step after components had already been integrated. I had the false assumption that due to the prototype nature of the project, that writing tests as I developed would be a waste of time, and that I should focus instead on feature development. This was a mistake, I have learnt that writing tests for components massively speeds up debugging and also catches bugs that escape manual testing. Had unit tests been written for components before their integration, bugs would have been caught at the component level and been easier to fix.

A.2 Personal Reflection and Lessons Learned

From the outset I understood that this project would be a huge challenge both conceptually with the theory it is based on and the volume and complexity of the code that would be required to implement it. I am proud of myself for choosing such a challenge and pushing myself to learn so many new things. When I started this project I had never heard of iso-surfaces let alone feature level-sets, did not have much experience with C++ and was only half way through the Graphics module which serves as an introduction to OpenGL and graphics programming. I knew that by pursuing this project I would have the unique opportunity learn about this interesting area of research, about graphics programming and become a better C++ programmer, and I feel I have achieved all of that.

With support from my supervisor I have also learnt to trust in my own judgement, I have learnt to not seek validation for each decision I make. I also focus on not asking questions to validate I am correct, but to reserve questions for when I need another person's expertise and to ensure I have given adequate thought before they are asked.

A.3 Legal, Social, Ethical and Professional Issues

A.3.1 Legal Issues

The only legal issues surrounding this project are the use of materials that I did not author, below are a list of these and how I have remained in compliance with their licenses.

- **Hurricane Isabel Dataset:** This dataset has no restrictions on its use, however, it is asked that if it is used, an attribution is included, which we have included in the acknowledgements section of this report.
- **Glad library and Khronos Platform header file:** The Khronos Platform header file is file needed for the Glad library to work. Both use a free of charge license (with Glad using the MIT License) without any limitations, subject to appropriate copyright attributions which are included in the source files which are used in the project (`third_party/glad`).
- **GLFW:** This library is licensed under the zlib/libpng license, meaning the software is free to use for any purpose, as long its origin is not misrepresented and any altered versions are plainly marked. GLFW has been used in this project in compliance with this license.
- **Dear ImGui:** This library uses the the MIT License, to comply with this the license file is included in `third_party/imgui/LICENSE.txt`.
- **GoogleTest:** This library is included in this project as a git submodule, meaning we simply store a reference to the source repository and have no control of the code. A license file (LICENSE) is available in the top level directory of the GoogleTest repository and this project complies with it.
- **OpenSans:** This font is used in the GUI and is included in `assets/fonts`, it ss covered by the Open Font License. To comply with this license it is included with the font in `LICENSE.txt`.

A.3.2 Social Issues

In this report the Hurricane Isabel dataset was used as an example use case. It is important when using real-world data in research to not ignore the need for respect and sensitivity to those the event affected. Hurricane Isabel was a category 5 major hurricane which resulted in 51 fatalities in the United States and cost in excess of \$5 billion USD [18]. It is crucial therefore, that our use of this dataset in visualisation and analysis is conducted with the utmost care and responsibility.

A.3.3 Ethical Issues

Given this software is designed to help researchers extract features from generic datasets, it inherently lacks restrictions regarding its application or the types of datasets can be processed. This flexibility, while core to the aim of the project, opens up the possibility for unethical use. However, I believe that its flexibility is core to its potential utility across various disciplines and that the responsibility for the ethical use of a tool rests not with the tool or its creators but with the user.

A.3.4 Professional Issues

Industry standard practices were carried out in the development of this project, the code employs a consistent naming scheme, organised structure and is well commented. Version control was used throughout and where appropriate unit tests ensure the independent functionality of the project's components.

All external materials used in this project have been credited and this project includes all external material's licenses.

Appendix B

External Material

Here is a list of all external materials used, the licensing and permission for the usage of this material has been discussed in A.3.1. The justification for the usage of these external materials can be found in 2.2.

- **Hurricane Isabel Dataset:** The dataset used to as a case study for the usage of software produced by this project.
- **Glad:** The function loader library we chose to facilitate OpenGL development.
- **GoogleTest:** The testing library used for our unit tests.
- **Dear ImGui:** The GUI library which we chose to use for this project.
- **GLFW:** The library we use to manage OpenGL contexts, windowing and capturing keyboard and mouse input.
- **GLM:** The library we use for matrix and vector operations.
- **OpenSans Font:** The font we use for our GUI.

Appendix C

UML Diagrams

This appendix contains UML diagrams for the classes referenced in chapter 3.

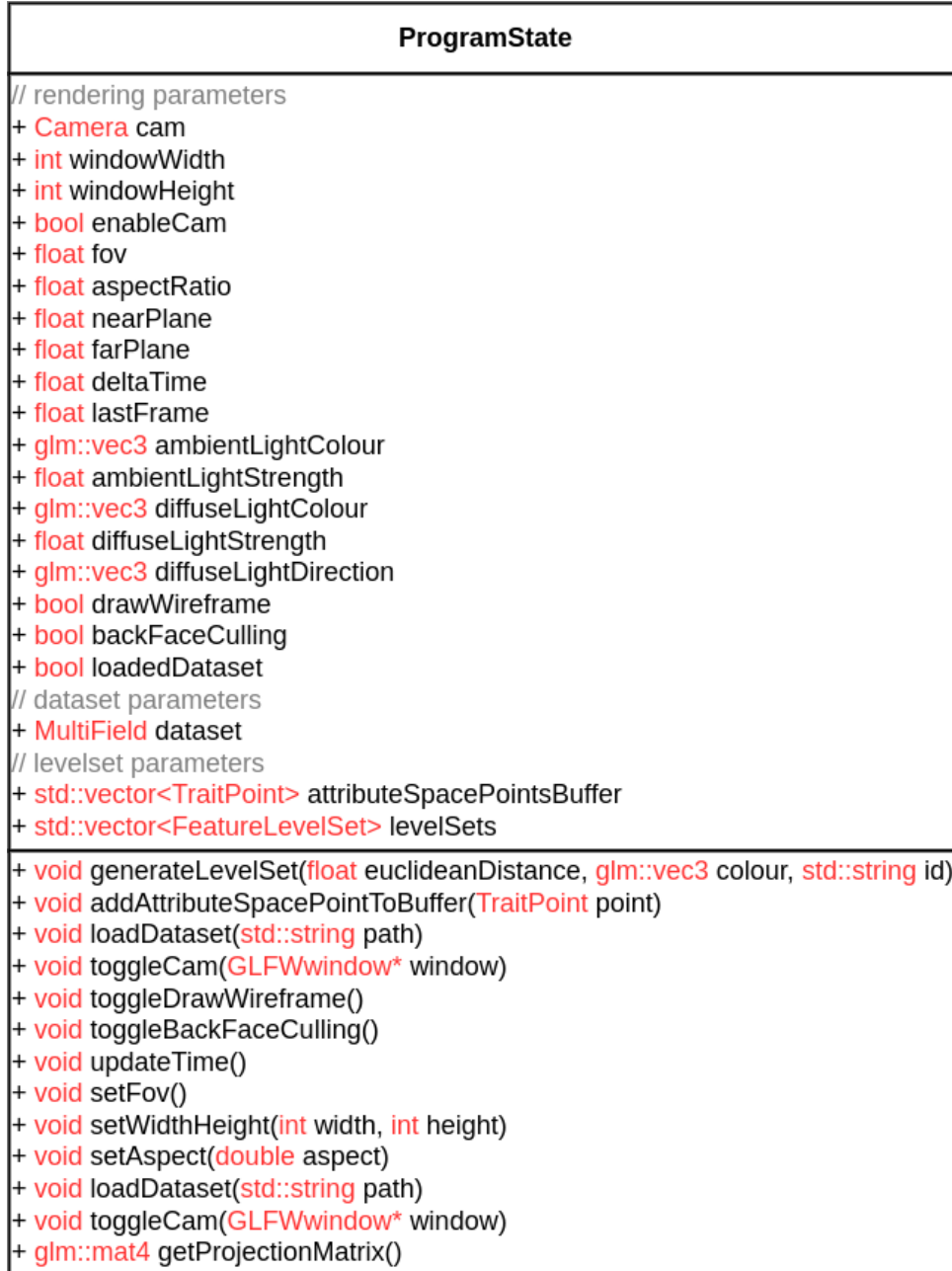


Figure C.1: Class diagram for ProgramState, which is defined in src/main/state.hpp

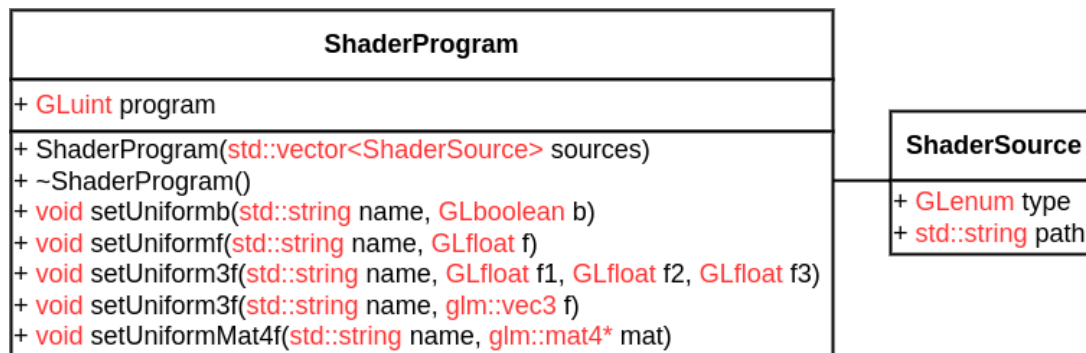


Figure C.2: Class diagrams for `ShaderProgram` and `ShaderSource`, which are defined in `src/lib/shaderTools/shaderProgram.hpp`

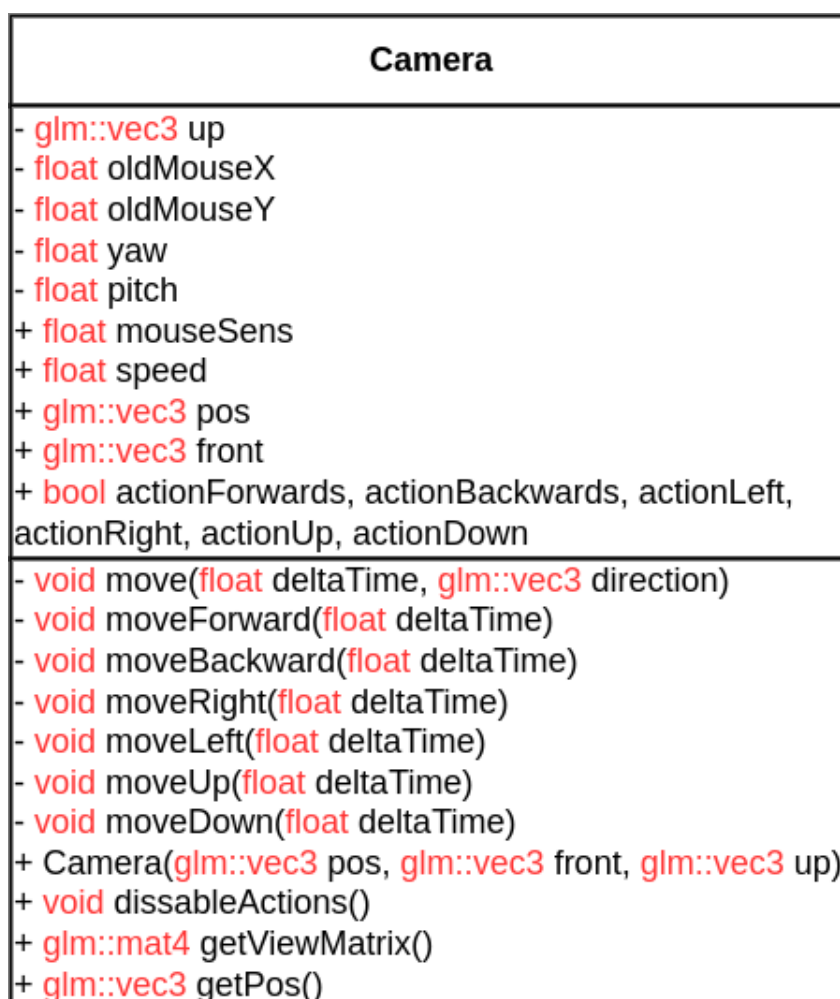


Figure C.3: Class diagram for `Camera`, which is defined in `src/lib/camera/camera.hpp`

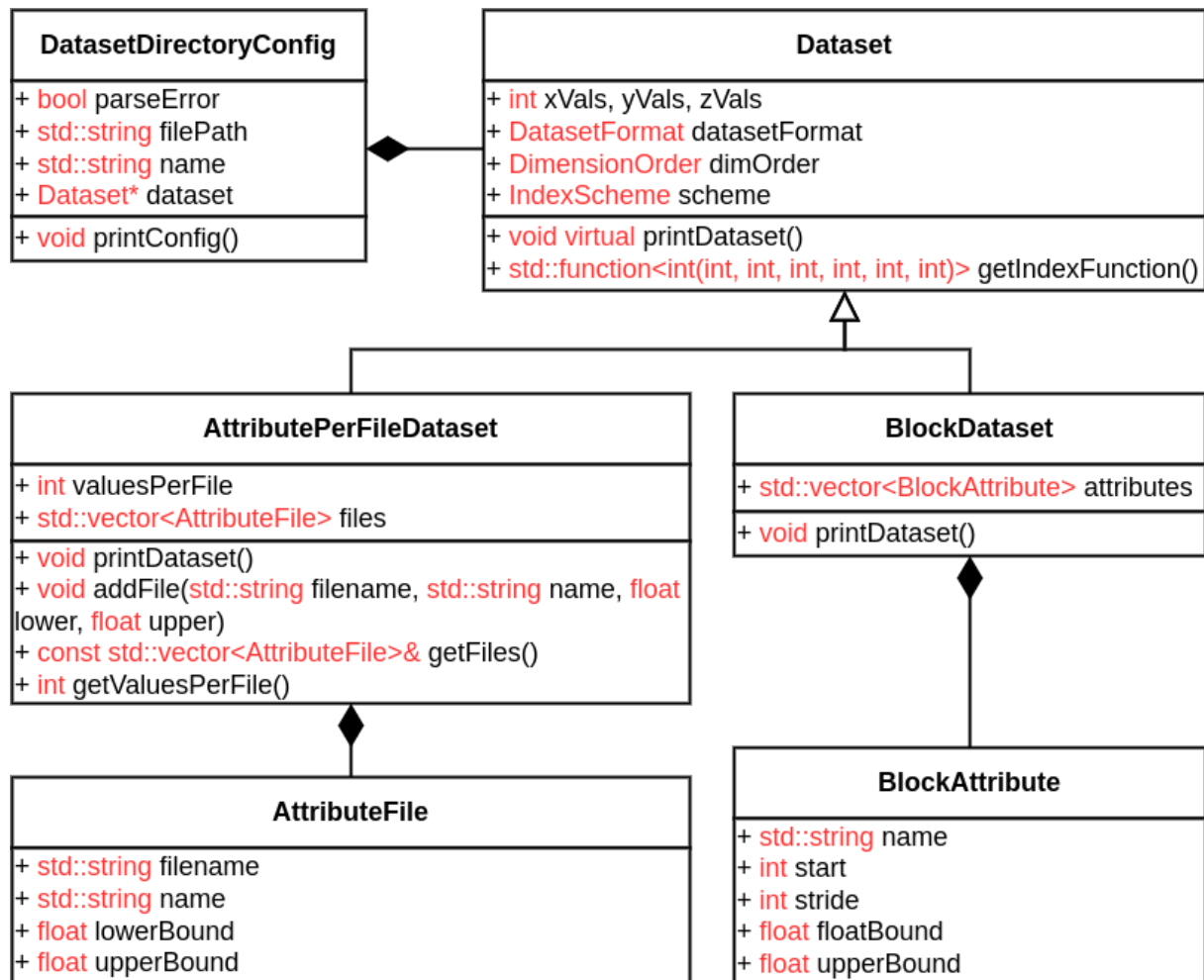


Figure C.4: Class diagrams for classes which are used by the dataset reader and defined in `src/lib/parsingData/datasetConfig.hpp`

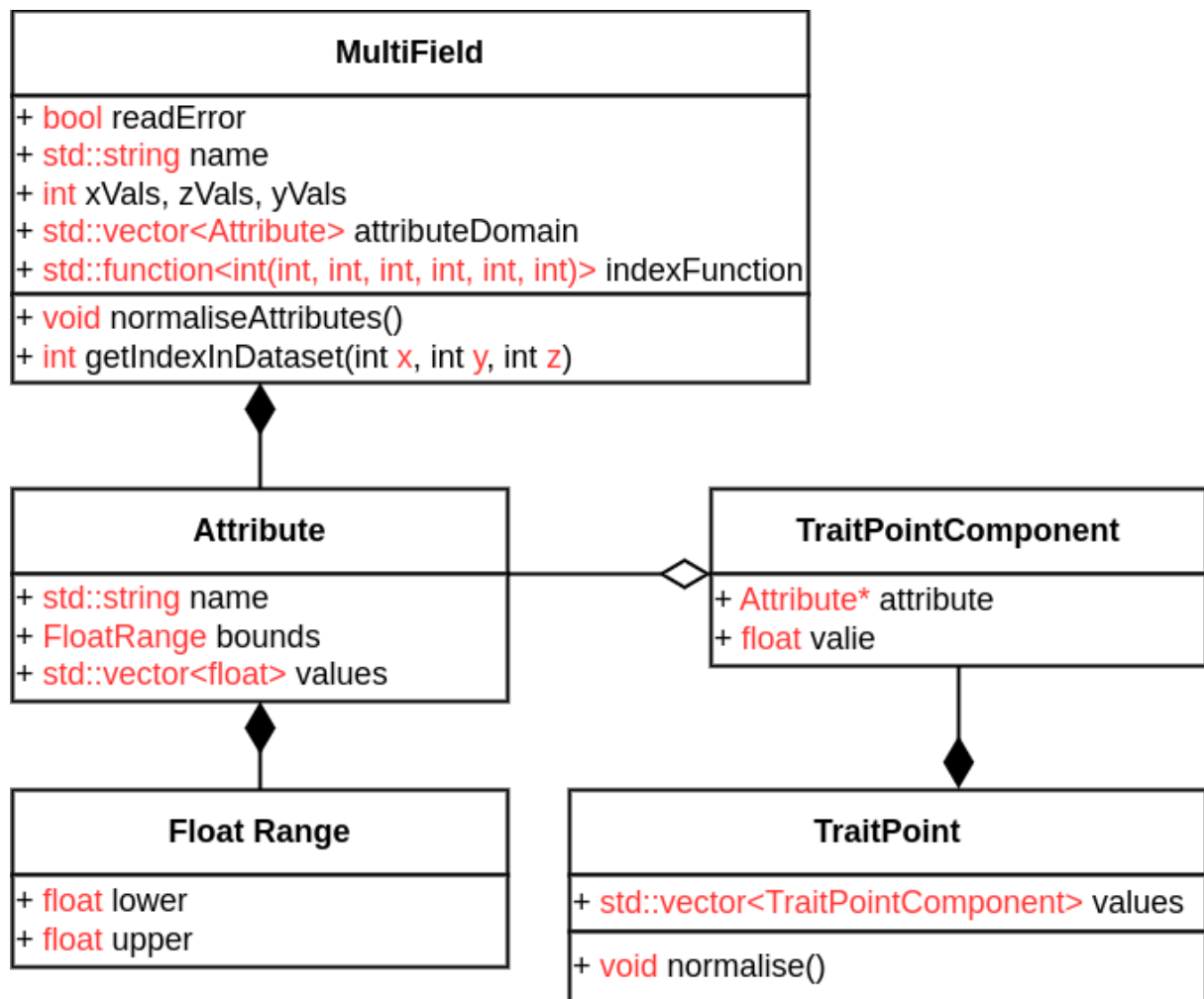


Figure C.5: Class diagrams for the MultiField and related classes, which are defined in `src/lib/multiField/multiField.hpp`

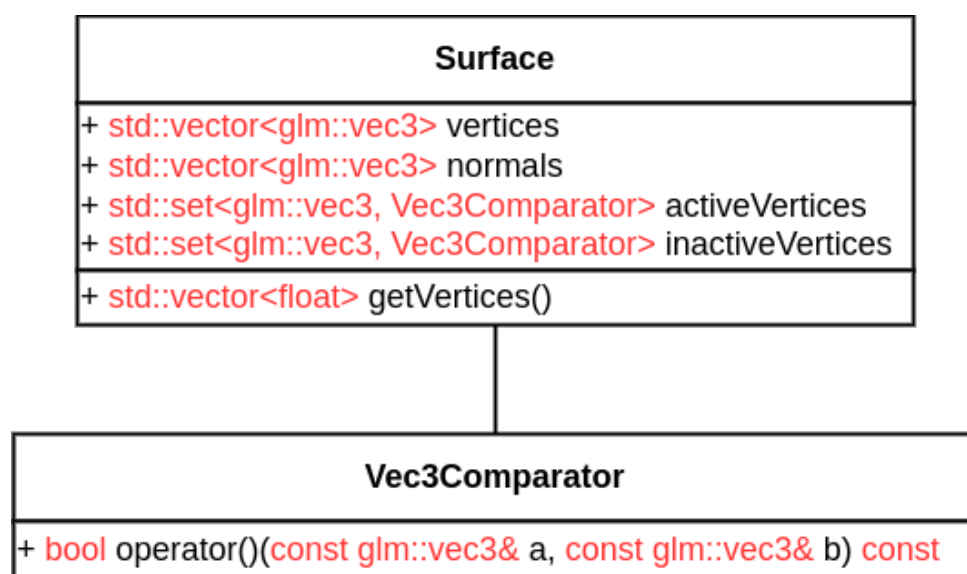


Figure C.6: Class diagrams for the Surface and Vec3Comparator, which are both defined in `src/lib/marchingCubes/marchingCubes.hpp`

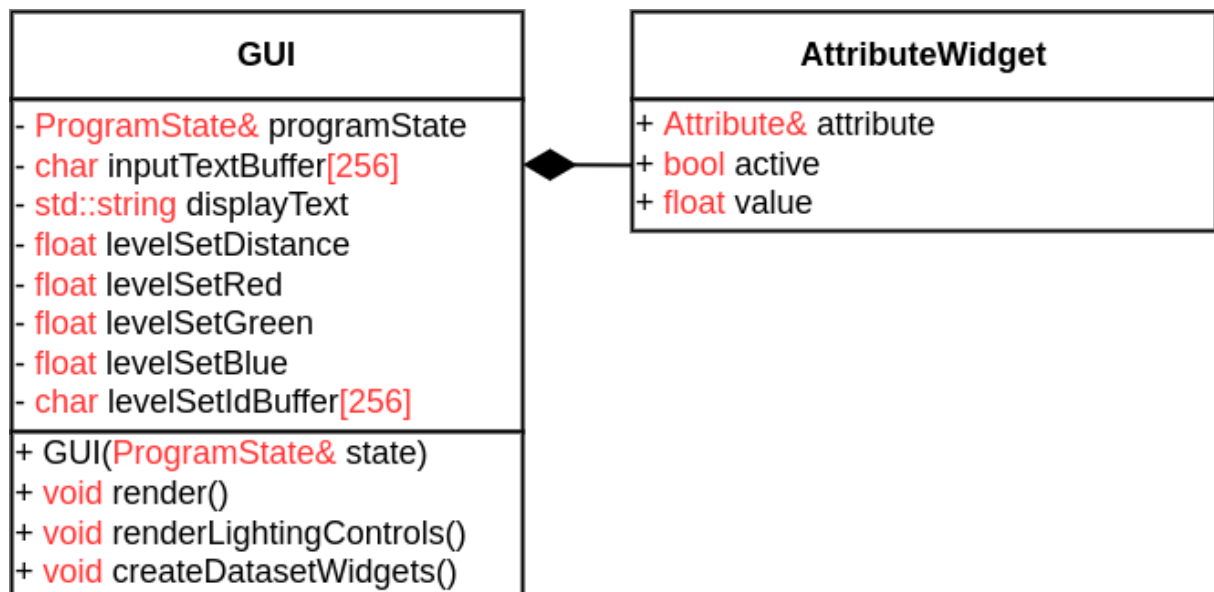


Figure C.7: Class diagrams for the GUI and AttributeWidget, which are both defined in src/lib/gui/gui.hpp

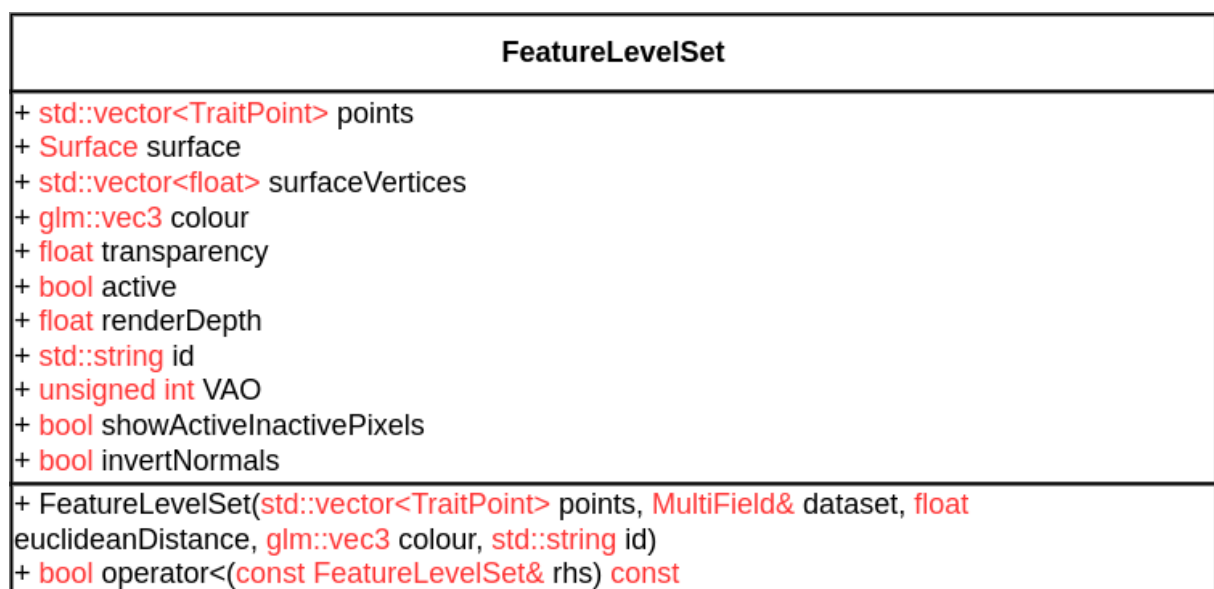


Figure C.8: Class diagram for FeatureLevelSet, defined in src/lib/levelSets/featureLevelSet.hpp

Appendix D

Code Snippets

This appendix includes code used in the project that is not appropriate to include in the project repository, this is code that is used to adapt and explore the Isabel dataset.

D.1 Generating the Velocity Derived Attribute

The compute velocity python function generates a derived attribute velocity, which is the magnitude of the wind velocity taking its three components, x-direction, y-direction and z-direction.

```
1 import numpy as np
2 import os
3
4 def compute_velocity(folder_path):
5     # get the current timestep
6     timestep = folder_path.split("/timestep")[-1]
7
8     # File names based on provided n
9     u_file = os.path.join(folder_path, f"Uf{timestep}.bin")
10    v_file = os.path.join(folder_path, f"Vf{timestep}.bin")
11    w_file = os.path.join(folder_path, f"Wf{timestep}.bin")
12    output_file = os.path.join(folder_path, "velocity.bin")
13
14    # Read binary data from files
15    u = np.fromfile(u_file, dtype=np.float32).byteswap()
16    v = np.fromfile(v_file, dtype=np.float32).byteswap()
17    w = np.fromfile(w_file, dtype=np.float32).byteswap()
18
19    # Handle extreme values indicating missing wind speed
20    extreme_value = 1.0e35
21    u[np.abs(u) >= extreme_value] = np.nan
22    v[np.abs(v) >= extreme_value] = np.nan
23    w[np.abs(w) >= extreme_value] = np.nan
24
25    # Calculate the magnitude of the velocity vector
26    velocity = np.sqrt(u**2 + v**2 + w**2)
27
28    # Reset NAN values
29    velocity[np.isnan(velocity)] = extreme_value
30
31    # Reverse the byte order for compatibility
32    velocity = velocity.byteswap()
33
34    # Write the result back to a binary file
35    velocity.tofile(output_file)
```

D.2 Generating Attribute Histograms

This is the python the code used to generate these histograms. The code removes the first bin of a histogram if this bin represents more than 70% of the histograms frequencies; this is useful for attributes such as total cloud cover where most points in the dataset have no cloud, and so the first bin is drowns out the rest of the histogram.

```

1  import numpy as np
2  import os
3  import matplotlib.pyplot as plt
4
5  # get the attribute data from the config file
6  def parse_config_file(config_path):
7      attribute_bounds = {}
8      with open(config_path, 'r') as file:
9          for line in file:
10             if line.startswith('ATTRIBUTE:'):
11                 parts = line.split(':')
12                 file_name = parts[1].strip()
13                 min_val = float(parts[3].strip())
14                 max_val = float(parts[4].strip())
15                 attribute_bounds[file_name] = (min_val, max_val)
16  return attribute_bounds
17
18
19 def output_histogram(folder_path):
20     # Parse the configuration file for bounds
21     config_path=os.path.join(folder_path, 'config.txt')
22     attribute_bounds = parse_config_file(config_path)
23
24     # Find all .bin files in the folder
25     files = [f for f in os.listdir(folder_path) if f.endswith('.bin')]
26
27     # number of subplots required, arranged in a grid
28     num_files = len(files)
29     rows = int(np.ceil(np.sqrt(num_files)))
30     cols = int(np.ceil(num_files / rows))
31
32     # plot dimensions and setup
33     plt.figure(figsize=(20, 10)) # Adjust the size as needed
34
35     for index, file in enumerate(files, 1):
36         file_path = os.path.join(folder_path, file)
37         data = np.fromfile(file_path, dtype=np.float32).byteswap()
38
39         # handle extreme values indicating missing data, by removing them
40         min_val, max_val = attribute_bounds.get(file, (None, None))
41         if min_val is not None and max_val is not None:
42             clean_data = data[(data >= min_val) & (data <= max_val)]
43
44         max_val = np.nanmax(clean_data)
45         min_val = np.nanmin(clean_data)

```

```

46
47     # check the first bin condition before plotting
48     counts, bin_edges = np.histogram(clean_data, bins=100)
49     if counts[0] / np.sum(counts) > 0.7:
50         first_bin_max = bin_edges[1]
51         clean_data = clean_data[clean_data > first_bin_max] # Exclude values that fall into the first bin
52
53     if data.size == 0:
54         continue # skip this file if there's no data to plot after removing first bin
55
56
57     # subplot for the current file
58     ax = plt.subplot(rows, cols, index)
59     ax.hist(clean_data, bins=100, color='red', alpha=1)
60     ax.set_title(file)
61     ax.set_xlabel('Attribute Value')
62     ax.set_ylabel('Frequency')
63     ax.grid(True)
64
65     # set x-axis limits exactly to the bounds if there are bounds
66     if min_val is not None and max_val is not None:
67         ax.set_xlim(min_val, max_val)
68
69     # print maximum velocity for current file
70     print(f"Max value in {file}: {max_val}")
71     print(f"Min value in {file}: {min_val}")
72
73     plt.tight_layout()
74     plt.show()

```
